

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA KỸ THUẬT ĐIỆN TỬ I



BÁO CÁO NGHIÊN CỨU KHOA HỌC

Đề tài: “Nghiên cứu thuật toán, thử nghiệm chức năng Return To Home cho robot.”

Giảng viên hướng dẫn : Ths. Phạm Văn Sự

Họ và tên sinh viên : Dương Sơn Quang – B22DCDT238

: Nguyễn Khắc Nhật Huy - B23DCDT123

Hệ đào tạo : Đại học chính quy

Mã số : 33-SV-2025-ĐT1

Hà Nội – 2025

LỜI CẢM ƠN

Trong thời gian thực hiện đề tài, em đã nhận được nhiều sự giúp đỡ, đóng góp ý kiến và chỉ bảo nhiệt tình của thầy cô và bạn bè. Với tình cảm chân thành nhất, cho phép em được bày tỏ lòng biết ơn sâu sắc đến tất cả những người đã tạo điều kiện, giúp đỡ trong quá trình học tập, nghiên cứu và hoàn thiện bài báo cáo.

Trước hết, em xin gửi lời cảm ơn đến Ban Giám đốc Học viện Công nghệ Bưu chính Viễn thông đã tạo cho em môi trường rèn luyện tốt để em có thể học tập và tiếp thu những kiến thức trong những năm là sinh viên.

Em cũng xin chân thành cảm ơn tất cả các thầy cô giáo, đặc biệt là các thầy cô trong khoa Kỹ thuật điện tử 1 với tri thức, tâm huyết của mình đã truyền đạt vốn kiến thức và kinh nghiệm cho chúng em.

Em xin trân trọng cảm ơn thầy Phạm Văn Sự đã tận tâm giúp đỡ chỉ dạy những kiến thức quý báu để em có thể hoàn thành được đề tài này cũng như tiếp thu được những hành trang cần thiết để em có thể tiếp bước trên con đường sự nghiệp sau này.

Mục lục

LỜI CẢM ƠN	2
1 TỔNG QUAN VỀ CƠ SỞ LÝ THUYẾT	7
1.1 Tổng quan về IoT	7
1.1.1 Khái niệm IoT	7
1.1.2 Lịch sử hình thành	7
1.1.3 Kiến trúc của một hệ thống IoT	8
1.1.4 Ứng dụng của IoT	8
1.1.5 Ưu và nhược điểm của IoT	9
1.2 Tổng quan về robot điều khiển	9
1.2.1 Khái niệm	9
1.2.2 Các loại robot điều khiển phổ biến	9
1.2.3 Các thành phần chính của một robot điều khiển	9
1.2.4 Lợi ích của robot điều khiển	10
1.3 Phần mềm PlatformIO	11
1.3.1 Giới thiệu	11
1.3.2 Ưu điểm	11
1.4 ESP32	12
1.4.1 Giới thiệu	12
1.4.2 Thông số kỹ thuật	12
1.4.3 Sơ đồ chân ESP32	13
1.5 ESP32-S3	14
1.5.1 Giới thiệu	14
1.5.2 Thông số kỹ thuật	14
1.5.3 So sánh ESP32 và ESP32S3	14
1.5.4 Ứng dụng	14
1.6 Mạch điều khiển động cơ DC L298N	15
1.6.1 Tổng quan	15
1.6.2 Cấu tạo và hoạt động	15
1.6.3 Thông số kỹ thuật	15
1.6.4 L298N gồm các chân	15
1.6.5 Ưu điểm	16
1.6.6 Ứng dụng	17
1.7 Module GPS NEO-6M	17
1.7.1 Giới thiệu	17
1.7.2 Thông số kỹ thuật	18
1.7.3 Định dạng dữ liệu NMEA	18
1.7.4 Thư viện TinyGPS++	19

1.7.5	Lưu ý khi sử dụng	20
1.8	Cảm biến MPU9250	20
1.8.1	Giới thiệu	20
1.8.2	Thông số kỹ thuật	21
1.8.3	Ứng dụng	21
1.8.4	Thư viện MPU9250 _{asukiaaa}	21
1.9	Module nRF24L01	22
1.9.1	Giới thiệu	22
1.9.2	Thông số kỹ thuật	22
1.9.3	Ưu điểm	23
1.9.4	Kiến trúc Data Pipe	23
1.9.5	Ứng dụng trong dự án	24
1.10	Cảm biến siêu âm HC-SR04	24
1.10.1	Nguyên lý hoạt động	24
1.10.2	Thông số kỹ thuật	24
1.10.3	Lưu ý khi sử dụng	25
1.10.4	Ứng dụng trong dự án	26
1.11	Màn hình OLED SSD1306	26
1.11.1	Giới thiệu	26
1.11.2	Thông số kỹ thuật	26
1.11.3	Ứng dụng trong đề tài	26
1.12	Module joystick PS2	27
1.12.1	Giới thiệu	27
1.12.2	Thông số kỹ thuật	27
1.13	Module QMC5883L	27
1.13.1	Giới thiệu	27
1.13.2	Thông số kỹ thuật	27
1.13.3	Nguyên lý đo từ trường	28
1.13.4	Bù nghiêng	28
1.13.5	Calibration (Hiệu chỉnh)	29
1.13.6	Ưu và nhược điểm	30
2	THIẾT KẾ HỆ THỐNG	35
2.1	Mô tả hoạt động của hệ thống	35
2.1.1	Tổng quan	35
2.1.2	Chức năng chính	36
2.2	Kiến trúc FreeRTOS	38
2.3	Hệ thống tay cầm điều khiển (Controller)	39
2.3.1	Tổng quan	39
2.3.2	Sơ đồ khối	41
2.3.3	Mô tả hoạt động chi tiết	41
2.3.4	Các cơ chế hoạt động	46
2.3.5	Bảng so sánh các chế độ	53
2.3.6	Xử lý lỗi và failsafe	53
2.4	Sơ đồ khối hệ thống	54
2.5	Luồng dữ liệu Return To Home	55
2.6	Thuật toán Return To Home	56
2.6.1	Tổng quan	56

2.6.2	Các bước chi tiết	56
2.6.3	Độ chính xác và hạn chế	58
3	LẬP TRÌNH VÀ THỬ NGHIỆM	59
3.1	Code Controller (ESP32)	59
3.1.1	Cấu trúc code	59
3.1.2	Xử lý Joystick	59
3.1.3	Button State Machine (FSM)	60
3.1.4	Gửi lệnh qua nRF24L01	60
3.2	Robot	60
3.2.1	Kiến trúc FreeRTOS	60
3.2.2	TaskSensor - Đọc cảm biến	61
3.2.3	TaskRF - Nhận lệnh RF	62
3.2.4	TaskControl - State Machine + PID	63
3.2.5	TaskUI - Hiển thị OLED	65
3.2.6	TaskStorage - Ghi NVS	67
3.3	Kiến trúc FreeRTOS	67
3.3.1	Lợi ích của FreeRTOS	68
3.3.2	Pinning Tasks to Cores	68
3.3.3	Sơ đồ luồng dữ liệu	69
3.4	Các cải tiến quan trọng trong phiên bản mới	69
3.4.1	Tách riêng la bàn điện tử QMC5883L	69
3.4.2	Bộ lọc trung bình trượt (Moving Average Filter)	69
3.4.3	Cải tiến thuật toán tránh vật cản	70
3.4.4	Chức năng hiệu chuẩn từ kế (Magnetometer Calibration)	72
3.4.5	Cải tiến xử lý RF	74
3.4.6	Cải tiến PID Controller	76
3.4.7	Failsafe cải tiến	77
3.5	Code Controller (ESP32)	77
3.5.1	Thêm chức năng Triple-Click	77
3.5.2	Tank Control Logic	78
3.6	Bảng so sánh phiên bản cũ và mới	80
3.7	TaskUI - Hiển thị trạng thái hiệu chuẩn	80
3.8	Công thức toán học	82
3.8.1	Công thức bù nghiêng (Tilt Compensation)	82
3.8.2	Công thức Haversine	82
3.8.3	Công thức Bearing	83
3.9	Kết quả thử nghiệm	83
3.9.1	Test hiệu chuẩn từ kế	83
3.9.2	Test bộ lọc Moving Average	83
3.9.3	Test tránh vật cản cải tiến	83
3.9.4	Test RTH từ 20m	84
3.10	Hướng dẫn sử dụng	84
3.10.1	Khởi động hệ thống	84
3.10.2	Hiệu chuẩn từ kế (bắt buộc lần đầu)	84
3.10.3	Lưu vị trí Home	85
3.10.4	Kích hoạt RTH	85
3.10.5	Xóa Home	85

3.10.6	Hủy RTH giữa chừng	85
3.11	Xử lý lỗi thường gặp	85
3.11.1	Robot không nhận lệnh từ controller	85
3.11.2	Heading không chính xác	86
3.11.3	GPS không fix	86
3.11.4	Robot bị kẹt khi tránh vật cản	86
3.11.5	Watchdog reset liên tục	86
4	KẾT LUẬN	87
4.1	Hình ảnh sản phẩm	88
4.2	Kết quả đạt được	94
4.3	Kiến thức thu được	94
4.4	Ứng dụng thực tế	95
4.5	Hướng phát triển	95
4.5.1	Ngắn hạn	95
4.5.2	Dài hạn	95
4.6	Lời kết	95
4.7	Một số hình ảnh trong quá trình thực hiện đề tài	96
	TÀI LIỆU THAM KHẢO	98

Chương 1

TỔNG QUAN VỀ CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về IoT

1.1.1 Khái niệm IoT

IoT viết tắt của thuật ngữ “Internet of Thing” đề cập đến hàng tỷ thiết bị vật lý được kết nối thông qua internet, thu thập và chia sẻ dữ liệu. Với IoT, các thiết bị thông minh có thể giao tiếp với nhau và với các hệ thống điều khiển từ xa, tạo ra một hệ sinh thái kết nối thông minh, đem lại nhiều tiện ích cho con người. Các thiết bị IoT bao gồm các cảm biến, máy tính nhúng, các thiết bị điện tử gia đình, ô tô tự lái, nhà thông minh, các thiết bị giám sát an ninh. Các thiết bị có thể giao tiếp với nhau thông qua internet để thực hiện các chức năng đa dạng như giám sát và điều khiển, tự động hóa, phân tích dữ liệu, và cung cấp dịch vụ thông minh cho người sử dụng. Hệ thống IoT cho phép vật được cảm nhận hoặc được điều khiển từ xa thông qua hạ tầng mạng hiện hữu, tạo cơ hội cho thế giới thực được tích hợp trực tiếp hơn vào hệ thống điện toán, hệ quả là hiệu năng, độ tin cậy và lợi ích kinh tế được tăng cường bên cạnh việc giảm thiểu sự can dự của con người. Khi IoT được gia tổ cảm biến và cơ cấu chấp hành, công nghệ này trở thành một dạng thức của hệ thống ảo-thực với tính tổng quát cao hơn, bao gồm luôn cả những công nghệ như điện lưới thông minh, nhà máy điện ảo, nhà thông minh, vận tải thông minh và thành phố thông minh. Mỗi vật được nhận dạng riêng biệt trong hệ thống điện toán nhúng và có khả năng phối hợp với nhau trong cùng hạ tầng Internet hiện hữu.

Về cơ bản, Internet Vạn Vật cung cấp kết nối chuyên sâu cho các thiết bị, hệ thống và dịch vụ, kết nối này mang hiệu quả vượt trội so với kiểu truyền tải máy-máy (M2M), đồng thời hỗ trợ đa dạng giao thức, miền (domain), và ứng dụng. Kết nối các thiết bị nhúng này (luôn cả các vật dụng thông minh), được kỳ vọng sẽ mở ra kỷ nguyên tự động hóa trong hầu hết các ngành, từ những ứng dụng chuyên sâu như điện lưới thông minh, mở rộng tới những lĩnh vực khác như nhà thông minh, thành phố thông minh.

1.1.2 Lịch sử hình thành

Khái niệm về một mạng lưới thiết bị được kết nối với nhau đã được thảo luận vào đầu năm 1982, với một máy bán hàng tự động Coke được thực hiện ở Đại học Carnegie Mellon trở thành thiết bị kết nối Internet đầu tiên trên thế giới. Thuật ngữ “Internet of things”

- Quản lý các thiết bị cá nhân
- Đồng hồ đo thông minh

1.1.5 Ưu và nhược điểm của IoT

*Ưu điểm:

- Khả năng truy cập thông tin từ mọi nơi, mọi lúc trên mọi thiết bị.
- Cải thiện kết nối giữa các thiết bị điện tử.
- Chuyển các gói dữ liệu qua mạng được kết nối tiết kiệm thời gian và tiền bạc.
- Tự động hóa các nhiệm vụ giúp cải thiện chất lượng dịch vụ của doanh nghiệp và giảm nhu cầu can thiệp con người.

*Nhược điểm :

Vấn đề bảo mật khi có nhiều thiết bị kết nối

Việc thu thập dữ liệu là thách thức lớn. Nếu hệ thống trung tâm bị hỏng sẽ dẫn tới có khả năng các thiết bị kết nối sẽ bị ảnh hưởng.

Do không có tiêu chuẩn quốc tế nên dẫn tới các thiết bị từ các nhà sản xuất khác nhau không thể tương thích với nhau.

1.2 Tổng quan về robot điều khiển

1.2.1 Khái niệm

Robot điều khiển là những thiết bị có thể được điều khiển bởi con người để thực hiện các nhiệm vụ khác nhau. Chúng được trang bị các cảm biến, động cơ và một bộ điều khiển (thường là một vi điều khiển) để thực hiện các lệnh từ người điều khiển.

Robot có thể đáp ứng theo các ngữ cảnh thông minh một cách có chủ định theo thiết lập của người dùng, có thể hoạt động một cách tự động hoặc bán tự động, và có thể thay thế con người thực hiện một hoặc một số thao tác quản lý, điều khiển nhất định.

Robot điều khiển có nhiều ứng dụng và phổ biến: giải trí, giáo dục, nghiên cứu và ứng dụng thực tế

1.2.2 Các loại robot điều khiển phổ biến

- Robot điều khiển từ xa: Đặc điểm: Được điều khiển bằng một bộ điều khiển từ xa, có thể là tay cầm hoặc ứng dụng trên điện thoại. Ứng dụng: Chơi đồ chơi, quay phim, khám phá môi trường nguy hiểm. Ví dụ: Drone, xe điều khiển, robot hình người

- Robot tự động: Đặc điểm: Có khả năng tự động thực hiện các nhiệm vụ dựa trên các chương trình được cài đặt sẵn.

Ứng dụng: Hút bụi, cắt cỏ, giao hàng, khám phá hành tinh.

Ví dụ: Robot hút bụi Roomba, robot khám phá sao Hỏa.

1.2.3 Các thành phần chính của một robot điều khiển

Bộ điều khiển: Não bộ của robot, xử lý tín hiệu và điều khiển các động cơ.

Cảm biến: Giúp robot nhận biết môi trường xung quanh (ví dụ: cảm biến siêu âm, cảm biến ánh sáng, cảm biến nhiệt độ, camera...).

Động cơ: Cung cấp lực để robot di chuyển.

Bộ nguồn: Cung cấp năng lượng cho robot.



Hình 1.2: Robot điều khiển

Khung gầm: Cấu trúc cơ học của robot.

Cách thức hoạt động của Robot điều khiển bao gồm 4 bước: nhận tín hiệu – xử lý tín hiệu – thực hiện hành động – phản hồi.

1.2.4 Lợi ích của robot điều khiển

Robot điều khiển đem lại rất nhiều lợi ích cho con người như :

- Tiết kiệm thời gian, công sức.

- Thay thế con người làm những công việc nguy hiểm.

- Phục vụ tốt về vấn đề giải trí.

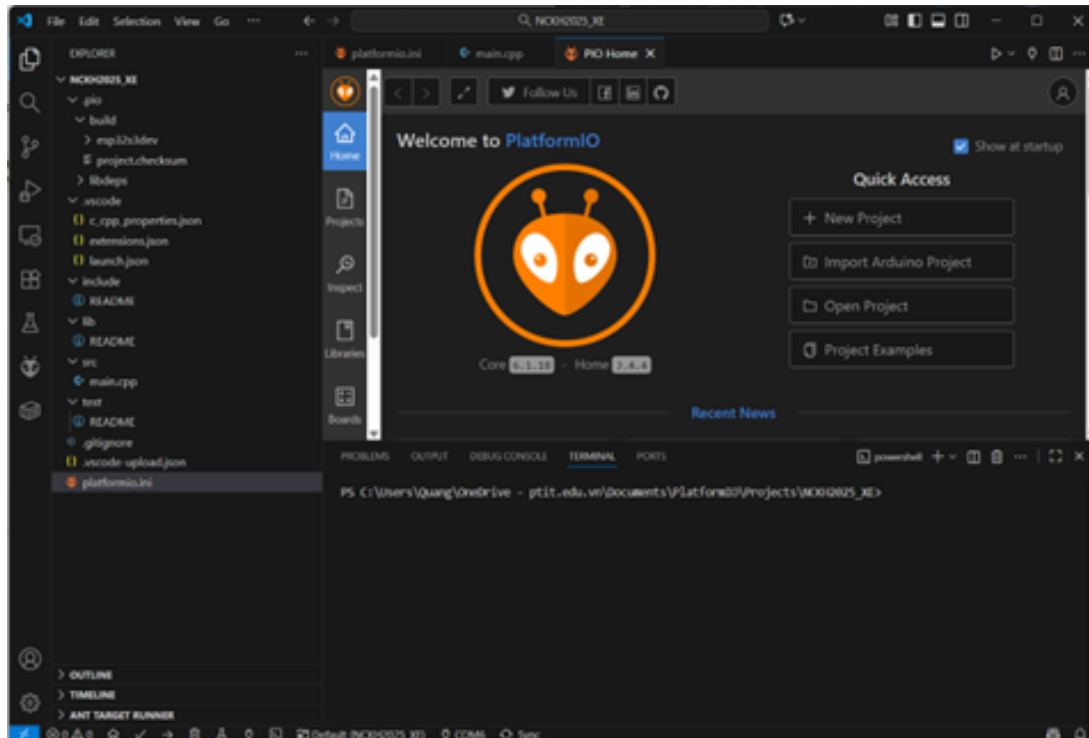
- Giúp học hỏi về khoa học, công nghệ.

Với sự phát triển của công nghệ, robot điều khiển ngày càng trở nên thông minh và đa năng hơn. Chúng sẽ được ứng dụng rộng rãi trong nhiều lĩnh vực của cuộc sống, từ gia đình đến công nghiệp.

1.3 Phần mềm PlatformIO

1.3.1 Giới thiệu

PlatformIO là một hệ sinh thái mã nguồn mở dành cho phát triển IoT (Internet of Things). Nó hỗ trợ hơn 800 board mạch khác nhau, bao gồm Arduino, ESP32, STM32, Raspberry Pi Pico và nhiều nền tảng khác.



Hình 1.3: Giao diện phần mềm PlatformIO

PlatformIO không phải là một IDE độc lập mà là một extension (tiện ích mở rộng) có thể tích hợp vào các IDE phổ biến như Visual Studio Code, Atom, CLion, Eclipse, Vim...

1.3.2 Ưu điểm

So với Arduino IDE:

Quản lý thư viện tốt hơn: Tự động tải và cập nhật thư viện, không cần cài đặt thủ công

Hỗ trợ nhiều nền tảng: Một IDE cho tất cả các board mạch Build system mạnh mẽ: Biên dịch nhanh hơn, hỗ trợ nhiều framework

IntelliSense: Tự động hoàn thành code, gợi ý hàm, phát hiện lỗi cú pháp

Debugging: Hỗ trợ debug trực tiếp trên board (với hardware debugger)

Unit Testing: Viết và chạy test tự động

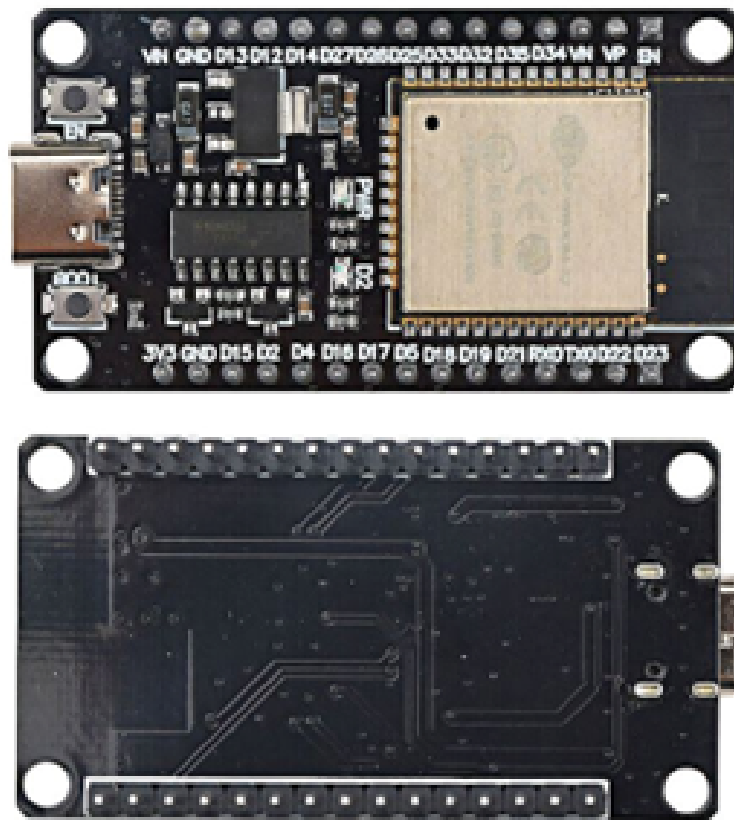
Quản lý project tốt: File platformio.ini chứa toàn bộ cấu hình project

CI/CD: Tích hợp dễ dàng với GitHub Actions, GitLab CI

1.4 ESP32

1.4.1 Giới thiệu

ESP32 là một loại vi điều khiển (microcontroller) vô cùng phổ biến và được yêu thích trong cộng đồng làm robot và IoT (Internet of Things). Nó được sản xuất bởi Espressif Systems và nổi bật với khả năng kết nối không dây mạnh mẽ, tích hợp cả Wi-Fi và Bluetooth. Điều này giúp ESP32 trở thành lựa chọn hàng đầu cho các dự án cần kết nối với internet hoặc các thiết bị khác một cách dễ dàng.



Hình 1.4: ESP 3232

1.4.2 Thông số kĩ thuật

Lõi xử lý: Xtensa LX6 32-bit, lõi đơn hoặc lõi kép.

Tần số: Lên đến 240 MHz.

Hiệu năng: Đạt 600 DMIPS.

Bộ đồng xử lý: Có bộ đồng xử lý công suất thấp (ULP) để thực hiện các tác vụ nền.

SRAM: 520 KB.

ROM: 448 KB.

Bộ nhớ Flash: Có thể lên đến 16 MB, tùy thuộc vào phiên bản.

Wi-Fi: Hỗ trợ chuẩn 802.11 b/g/n/e/i, tốc độ lên đến 150 Mbps.

Bluetooth: Hỗ trợ Bluetooth v4.2, BLE.

Các giao tiếp khác: SPI, I2C, UART, I2S, SDIO.

ADC: Đo tín hiệu analog.

DAC: Tạo tín hiệu analog.

Touch: Hỗ trợ cảm ứng.

Hall sensor: Cảm biến hiệu ứng Hall.

Điện áp hoạt động: 2.2V - 3.6V.

Tiêu thụ điện năng: Rất thấp, phù hợp cho các dự án chạy pin.

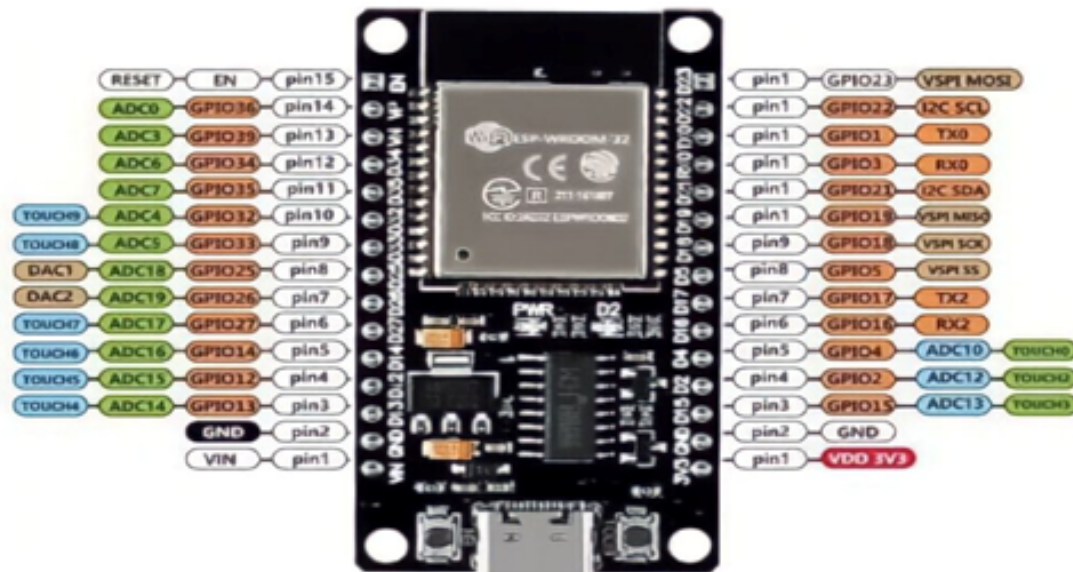
Hỗ trợ bảo mật: WPA/WPA2, SSL, TLS.

GPIO: Số lượng chân GPIO tùy thuộc vào phiên bản.

RTC: Đồng hồ thời gian thực.

Timer: Bộ hẹn giờ.

1.4.3 Sơ đồ chân ESP32



Hình 1.5: ESP32 pinout

1.5 ESP32-S3

1.5.1 Giới thiệu

ESP32-S3 là thế hệ vi điều khiển mới nhất của Espressif Systems, kế thừa và nâng cấp từ ESP32. ESP32-S3 được thiết kế đặc biệt cho các ứng dụng AI, IoT, và xử lý hình ảnh với hiệu suất cao hơn đáng kể so với các thế hệ trước.

1.5.2 Thông số kỹ thuật

Bộ xử lý:

- CPU: Xtensa LX7 dual-core 32-bit
- Tần số: Lên đến 240 MHz
- Hiệu năng: Cải thiện 15

Bộ nhớ:

- SRAM: 512 KB
- ROM: 384 KB
- Flash: Hỗ trợ lên đến 16 MB (tùy module)
- PSRAM: Hỗ trợ lên đến 8 MB (optional)

Kết nối không dây:

- Wi-Fi: 802.11 b/g/n (2.4 GHz)
- Bluetooth: Bluetooth 5 (LE)
- Tốc độ Wi-Fi: Lên đến 150 Mbps

Giao tiếp:

- 45x GPIO (Programmable)
- 2x UART
- 2x I2C
- 4x SPI
- 2x I2S
- USB OTG (Full-Speed)
- 20x ADC (12-bit)
- 2x DAC (8-bit)
- Touch sensor (14 kênh)

Tính năng đặc biệt:

- Hỗ trợ AI acceleration
- Camera interface (DVP 8/16-bit)
- LCD interface
- Bảo mật phần cứng (Secure Boot, Flash Encryption)
- RTC với Ultra-Low Power coprocessor

Điện năng:

- Điện áp hoạt động: 3.0V - 3.6V
- Chế độ Active: 40mA
- Chế độ Deep Sleep: <10μA

1.5.3 So sánh ESP32 và ESP32S3

1.5.4 Ứng dụng

- Robot tự động với AI

Bảng 1.1: So sánh ESP32 và ESP32-S3

Tính năng	ESP32	ESP32-S3
CPU	Xtensa LX6	Xtensa LX7 (+15% hiệu năng)
SRAM	520 KB	512 KB
Bluetooth	v4.2 BR/EDR + BLE	v5.0 LE only
USB	Không	USB OTG native
AI support	Không	Có (vector instructions)
Camera	Không native	DVP 8/16-bit native

- Camera giám sát thông minh (nhận diện khuôn mặt, biển số xe)
- Thiết bị IoT hiệu năng cao
- Voice assistant (nhận diện giọng nói)
- Thiết bị đeo thông minh
- Gateway IoT

1.6 Mạch điều khiển động cơ DC L298N

1.6.1 Tổng quan

L298N là một mạch tích hợp (IC) được sử dụng rộng rãi trong các dự án điện tử, đặc biệt là trong việc điều khiển động cơ DC. Nó đóng vai trò như một cầu H (H-bridge), cho phép thay đổi hướng quay và điều chỉnh tốc độ của động cơ

1.6.2 Cấu tạo và hoạt động

Cầu H: L298N tích hợp hai mạch cầu H độc lập, mỗi mạch có thể điều khiển một động cơ DC. Cầu H hoạt động bằng cách chuyển đổi cực của nguồn điện cung cấp cho động cơ, từ đó thay đổi hướng quay của động cơ.

Các chân điều khiển: Mỗi cầu H có các chân điều khiển tương ứng để xác định hướng quay và tốc độ của động cơ.

Chân cấp nguồn: Cần cung cấp nguồn điện một chiều cho cả mạch tích hợp và động cơ.

1.6.3 Thông số kĩ thuật

Driver: L298N tích hợp hai mạch cầu H.

Điện áp điều khiển: +5 V +12 V

Dòng tối đa cho mỗi cầu H là: 2A

Điện áp của tín hiệu điều khiển: +5 V +7 V

Dòng của tín hiệu điều khiển: 0 36mA (Arduino có thể chơi đến 40mA)

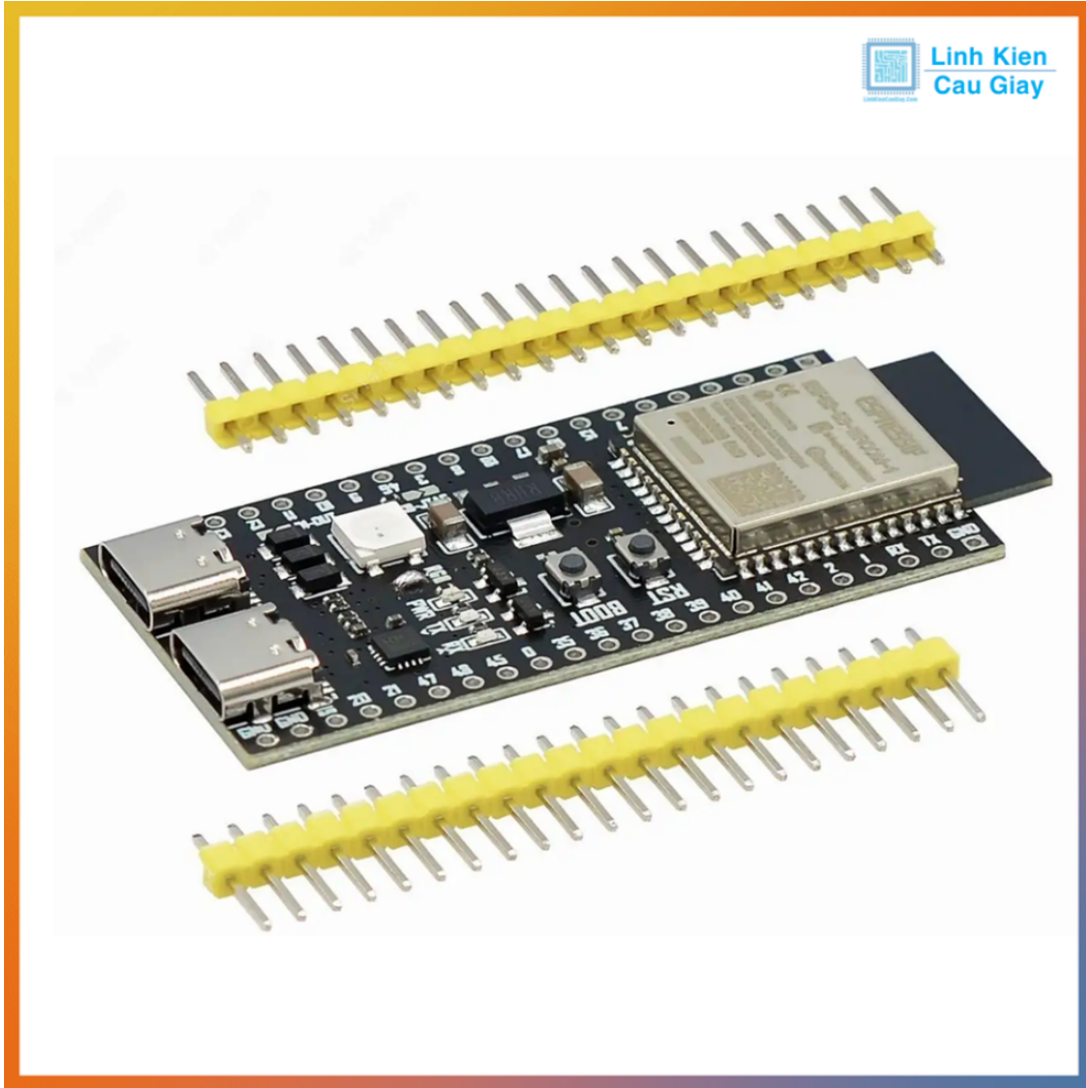
Công suất hao phí: 20W (khi nhiệt độ $T = 75^{\circ}\text{C}$)

Nhiệt độ bảo quản: -25°C $+130^{\circ}\text{C}$

1.6.4 L298N gồm các chân

12V power, 5V power. Đây là 2 chân cấp nguồn trực tiếp đến động cơ.

Có thể cấp nguồn 9-12V ở 12V.



Hình 1.6: ESP32-S3

Bên cạnh đó có jumper 5V, nếu cài jump vào 5V-EN để như hình ở trên thì sẽ có nguồn 5V ra ở cổng 5V power, ngược lại thì không. Jump đó giúp ta có nguồn 5v để nuôi những mạch khác(ví dụ như esp 32).

Power GND chân này là GND của nguồn cấp cho Động cơ.

2 Jump A enable và B enable.

Gồm có 4 chân Input. IN1, IN2, IN3, IN4.

Output A: nối với động cơ A.

Output B: nối với động cơ B.

*Vì là động cơ DC nên chú ý nối 2 cực + - của động cơ sao cho hợp lí.

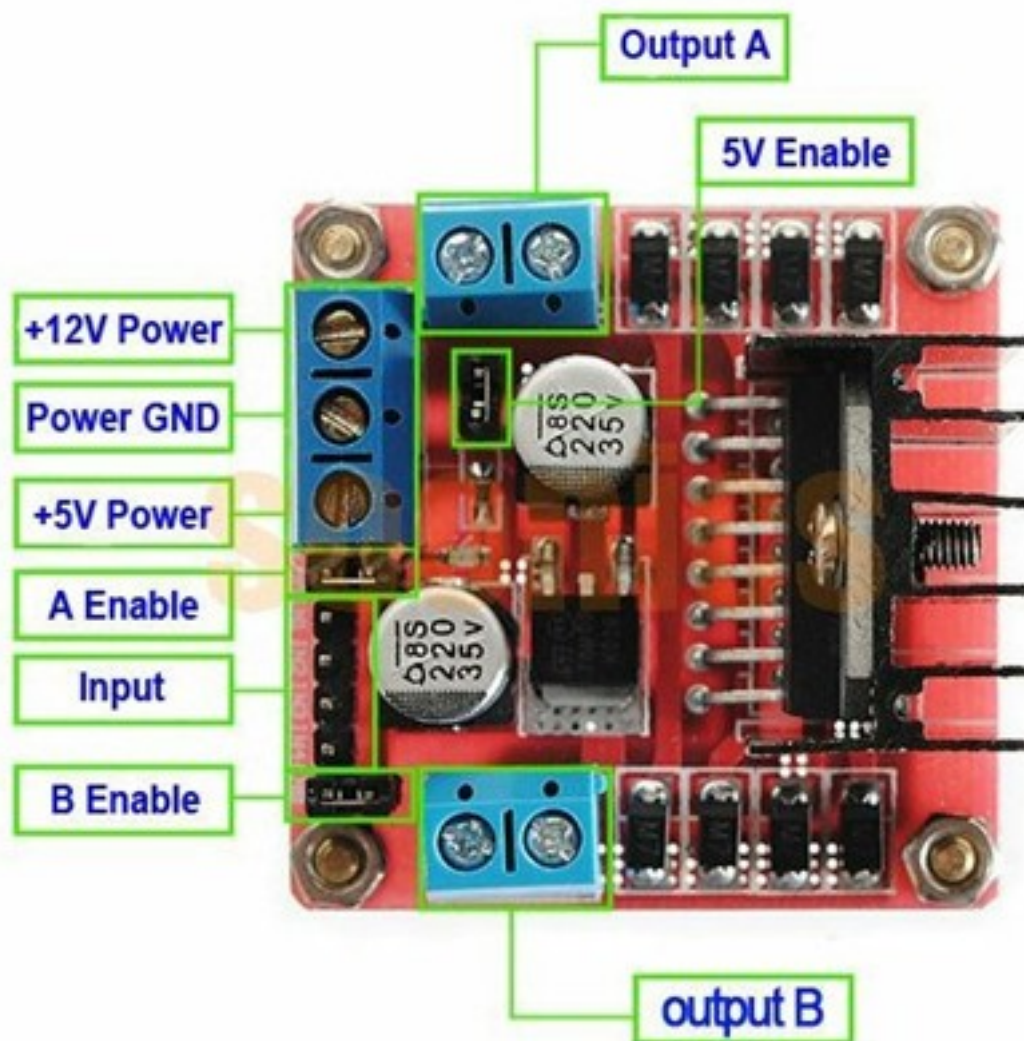
1.6.5 Ưu điểm

Đơn giản: Dễ sử dụng và kết nối.

Linh hoạt: Có thể điều khiển nhiều loại động cơ DC với công suất khác nhau.

Giá thành hợp lý: Dễ tiếp cận với các nhà làm dự án.

Ổn định: Hoạt động ổn định trong nhiều điều kiện làm việc khác nhau.



Hình 1.7: Mạch L298N

1.6.6 Ứng dụng

Robot: Điều khiển các động cơ của robot.

Xe tự hành: Điều khiển các động cơ của xe.

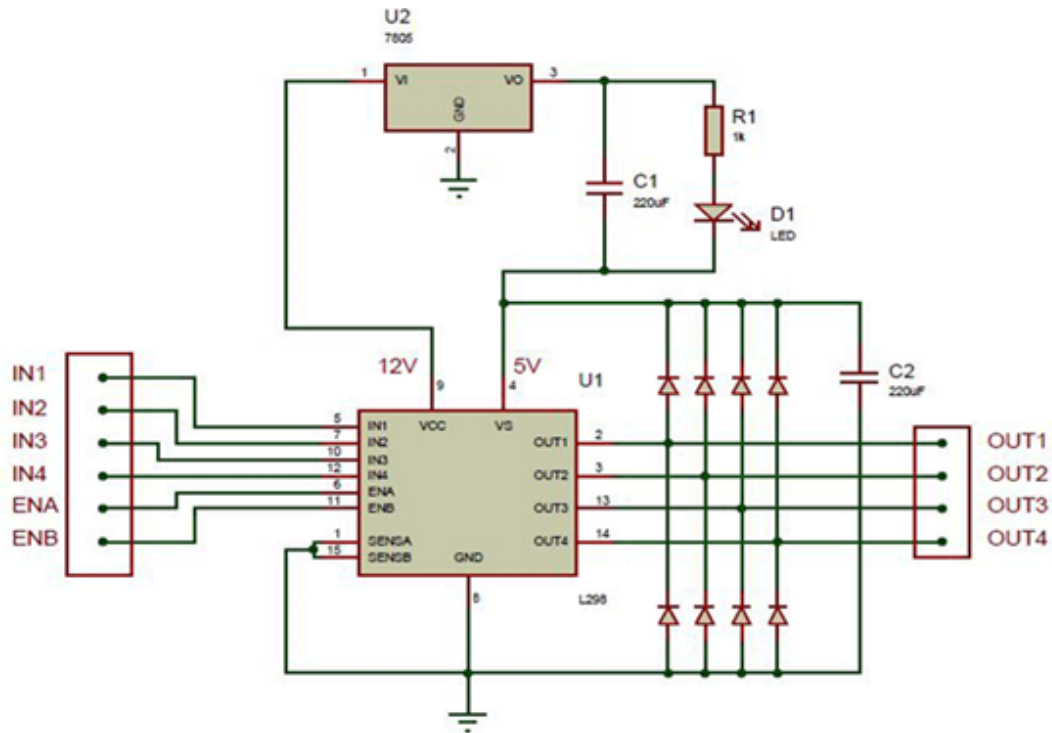
Hệ thống tự động hóa: Điều khiển các van, cửa, máy bơm.

Mô hình: Điều khiển các động cơ trong các mô hình tàu, máy bay.

1.7 Module GPS NEO-6M

1.7.1 Giới thiệu

NEO-6M là một module GPS (Global Positioning System) phổ biến của hãng u-blox, được sử dụng rộng rãi trong các dự án định vị, robot tự hành, drone, và các ứng dụng IoT.



Hình 1.8: L298N schematic

1.7.2 Thông số kĩ thuật

Thông số chính:

- Chip: u-blox NEO-6M
- Tần số: L1 (1575.42 MHz)
- Số kênh: 50 kênh
- Độ chính xác: 2.5m CEP (Circular Error Probable)
- Thời gian khởi động:
- Cold start: 27s
- Hot start: 1s - Tốc độ cập nhật: 5 Hz (mặc định 1 Hz)
- Độ nhạy:
- Tracking: -161 dBm
- Acquisition: -148 dBm - Điện áp: 3.3V 5V
- Dòng tiêu thụ: 45mA (tracking)

Giao tiếp:

- UART (TX, RX)
- Baud rate mặc định: 9600 bps
- Protocol: NMEA 0183, UBX binary

Antenna:

- Built-in ceramic antenna (hoặc có cổng kết nối antenna ngoài)

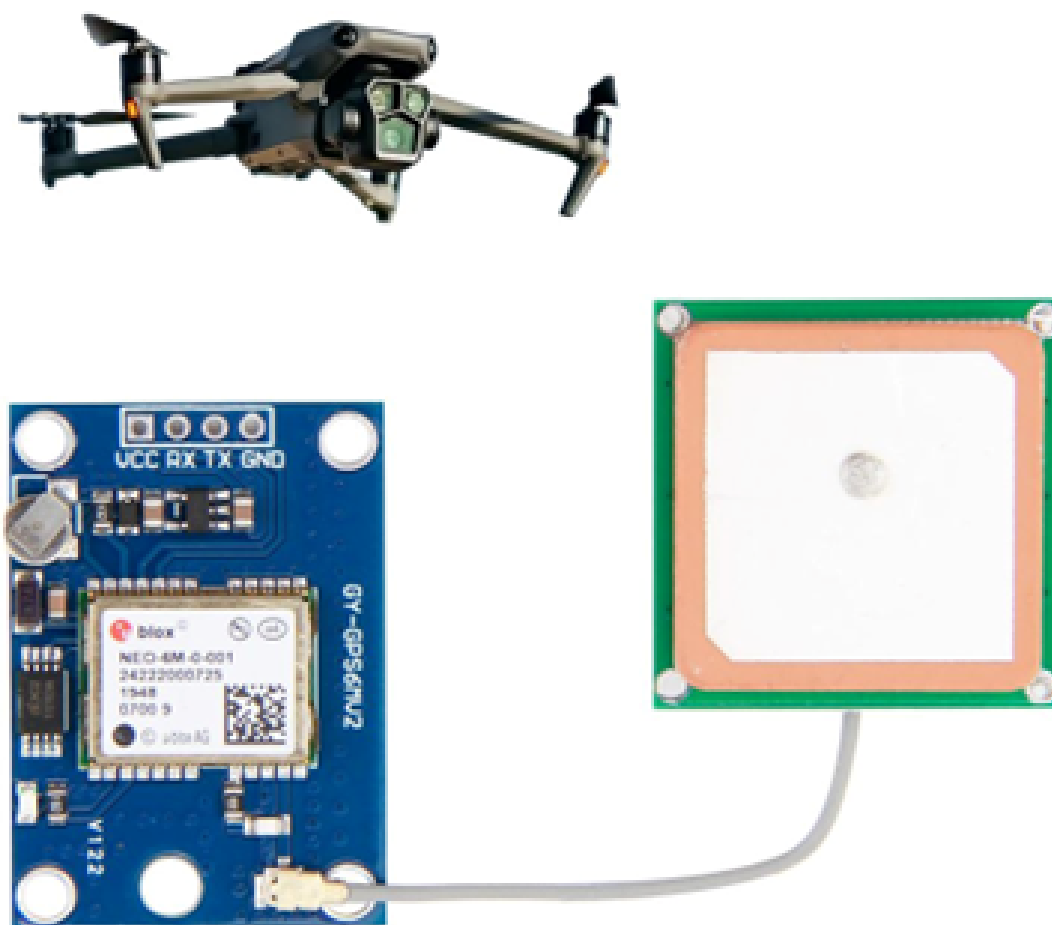
1.7.3 Định dạng dữ liệu NMEA

GPS NEO-6M truyền dữ liệu theo chuẩn NMEA 0183. Các câu lệnh phổ biến:

GPGGA: Thông tin vị trí cố định (Fix data)

GPGSA: Thông tin về độ chính xác (DOP)

GPGSV: Thông tin về vệ tinh nhìn thấy



Hình 1.9: Module NEO6M

GPRMC: Dữ liệu GPS tối thiểu (Recommended Minimum)

GPVTG: Vận tốc và hướng di chuyển

Ví dụ câu lệnh GPGGA:

GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

Trong đó:

- 123519: Thời gian UTC (12:35:19)
- 4807.038,N: Vĩ độ 48°07.038' Bắc
- 01131.000,E: Kinh độ 11°31.000' Đông
- 1: Fix quality (1 = GPS fix)
- 08: Số vệ tinh đang sử dụng
- 0.9: HDOP (Horizontal Dilution of Precision)
- 545.4,M: Độ cao so với mực nước biển (meters)

1.7.4 Thư viện TinyGPS++

TinyGPS++ là thư viện Arduino phổ biến để parse dữ liệu NMEA từ GPS. Ưu điểm:

- Nhẹ, hiệu quả

- Dễ sử dụng
- Hỗ trợ đầy đủ các câu lệnh NMEA
- Tính toán khoảng cách, góc bearing giữa 2 điểm GPS

```

1 // TinyGPS++ API
2 gps.location.lat()          // V
3 gps.location.lng()          // Kinh
4 gps.location.age()          // Thời gian t   l n   c p   n h t
   c u i (ms)
5 gps.location.isValid()      // Kiểm tra d   l i u   GPS   h p   l
6 gps.satellites.value()      // S   v   tinh
7 gps.speed.kmph()            // V n   t c (km/h)
8 gps.course.deg()            // H   ng   di   ch u y n (   )
9
10 // T n h   k h o n g   c   h   g i a   2   i m   (Haversine formula)
11 TinyGPSPlus::distanceBetween(lat1, lon1, lat2, lon2)
12
13 // T n h   g   c   bearing   t   i m   A   n   i m   B
14 TinyGPSPlus::courseTo(lat1, lon1, lat2, lon2)

```

1.7.5 Lưu ý khi sử dụng

Cần nhìn thấy bầu trời: GPS cần tín hiệu từ ít nhất 4 vệ tinh để có fix 3D (vĩ độ, kinh độ, độ cao). Không hoạt động trong nhà.

Cold start lâu: Lần đầu tiên hoặc sau khi mất điện lâu, GPS cần 27s để tải almanac từ vệ tinh.

Độ chính xác: 2.5m là lý thuyết, thực tế có thể dao động 5-10m tùy điều kiện. Tốc độ cập nhật: Mặc định 1 Hz (1 lần/giây), có thể tăng lên 5 Hz nếu cần.

Antenna: Module có antenna gồm tích hợp, nhưng nếu cần tầm xa hơn, dùng active antenna ngoài.

1.8 Cảm biến MPU9250

1.8.1 Giới thiệu

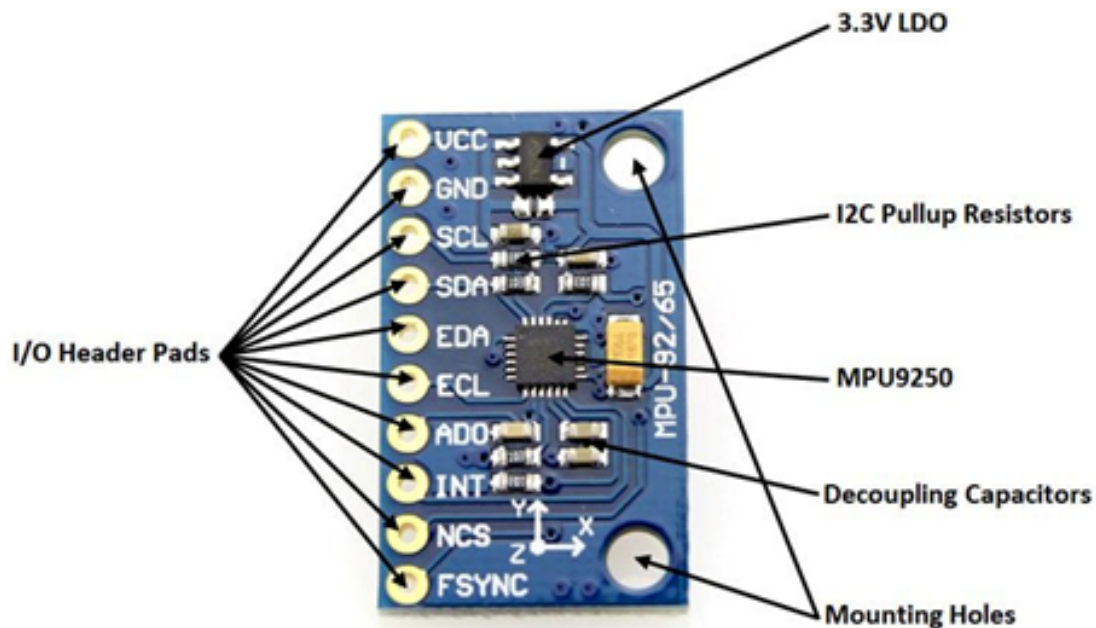
MPU9250 là một module IMU (Inertial Measurement Unit) 9 trục được phát triển bởi InvenSense (nay thuộc TDK). Đây là phiên bản nâng cấp của MPU6050 (6 trục) với việc tích hợp thêm từ kế (magnetometer).

9 trục gồm:

- 3 trục gia tốc kế (Accelerometer) - đo gia tốc theo X, Y, Z
- 3 trục con quay hồi chuyển (Gyroscope) - đo vận tốc góc
- 3 trục từ kế (Magnetometer) - đo từ trường (la bàn điện tử)

Trong dự án này, MPU9250 chỉ được sử dụng gia tốc kế (accelerometer) và con quay hồi chuyển (gyroscope) để tính toán góc nghiêng Roll và Pitch phục vụ cho việc bù nghiêng (tilt compensation) của la bàn điện tử.

Lưu ý: Từ kế (magnetometer) không dùng từ MPU9250 mà dùng module riêng QMC5883L (GY-271) để có độ chính xác cao hơn



Hình 1.10: MPU9250

1.8.2 Thông số kĩ thuật

- Cấu trúc : NEMS
- Tốc độ truyền SPI : 1Mhz
- Tốc độ đọc dữ liệu nhanh SPI : 20Mhz
- Tốc độ truyền I2C : 400 kHz
- Cổng SPI : 1
- Cổng I2C : 1
- Điện áp hoạt động : 2.4 - 3.6 V
- Dòng điện định mức : 3.5 mA

1.8.3 Ứng dụng

Tính góc nghiêng (Roll, Pitch): Từ dữ liệu gia tốc kế

- Tính hướng (Yaw/Heading): Từ từ kế (la bàn điện tử)
- Phát hiện chuyển động, rung động
- Bù nghiêng (Tilt Compensation): Hiệu chỉnh giá trị từ kế khi robot bị nghiêng
- Điều khiển drone, robot tự cân bằng
- Game controller, VR headset
- Hệ thống định hướng (AHRS - Attitude and Heading Reference System)

1.8.4 Thư viện MPU9250_{asukiaaa}

Đây là thư viện Arduino đơn giản và dễ sử dụng cho MPU9250:

```

1 #include
2 MPU9250_asukiaaa mpu;
3 mpu.setWire(&Wire);
4 mpu.beginAccel();
5 mpu.beginGyro();

```

```

6 mpu.beginMag();
7
8 // c d l i u
9 mpu.accelUpdate();
10 float ax = mpu.accelX(); // g
11
12 mpu.gyroUpdate();
13 float gx = mpu.gyroX(); // /s
14
15 mpu.magUpdate();
16 float mx = mpu.magX(); // T

```

1.9 Module nRF24L01

1.9.1 Giới thiệu

nRF24L01 là một module thu phát RF (Radio Frequency) không dây hoạt động ở tần số 2.4 GHz. Module này được sản xuất bởi Nordic Semiconductor và rất phổ biến trong các dự án Arduino, ESP32 nhờ giá thành rẻ, tầm xa, và tiêu thụ năng lượng thấp

1.9.2 Thông số kỹ thuật

RF:

- Tần số: 2.400 - 2.525 GHz (ISM band)
- Số kênh: 125 kênh (cách nhau 1 MHz)
- Tốc độ truyền: 250 kbps, 1 Mbps, 2 Mbps
- Công suất phát: 0 dBm, -6 dBm, -12 dBm, -18 dBm (tùy chỉnh)
- Độ nhạy thu: -94 dBm (@ 250 kbps)
- Tầm xa:
- Không có PA/LNA: 100m (ngoài trời)
- Có PA/LNA (nRF24L01+PA+LNA): 1000m

Giao tiếp:

- SPI: Tốc độ lên đến 10 Mbps
- Chân: CE, CSN, SCK, MOSI, MISO, IRQ (optional)

Nguồn:

- Điện áp: 1.9V - 3.6V (Module breakout thường có bộ ổn áp 3.3V)
- Dòng tiêu thụ:
- TX: 11.3 mA (@ 0 dBm)
- RX: 13.5 mA
- Standby: 26 μ A
- Power down: 900 nA

Packet:

- Payload: 1 - 32 bytes
- Hỗ trợ Auto-ACK (xác nhận tự động)
- Hỗ trợ Auto-Retransmit (gửi lại tự động nếu mất ACK)



Hình 1.11: Module nRF24L01

1.9.3 Ưu điểm

- Giá rẻ (10,000 VND/module)
 - Tầm xa tốt (với PA/LNA lên đến 1 km)
 - Tiêu thụ năng lượng thấp
 - Hỗ trợ nhiều kênh (tránh nhiễu)
 - Có Auto-ACK và Auto-Retransmit (tăng độ tin cậy)
 - Dễ sử dụng với thư viện RF24
 - có thư viện phổ biến, dễ sử dụng (RF24)

1.9.4 Kiến trúc Data Pipe

nRF24L01 hỗ trợ 6 data pipe (đường ống dữ liệu):

- Pipe 0: Dùng cho Auto-ACK (tự động)
- Pipe 1-5: Có thể cấu hình địa chỉ riêng

Mỗi pipe có địa chỉ 3-5 bytes (giống địa chỉ MAC). Hai module muốn giao tiếp phải dùng cùng địa chỉ.

Ví dụ:

```
1 const byte address[6] = "00001"; // a c h 5 bytes
2
3 // Module A (transmitter):
4 radio.openWritingPipe(address);
5
6 // Module B (receiver):
7 radio.openReadingPipe(1, address);
```

1.9.5 Ứng dụng trong dự án

Trong dự án robot RTH, nRF24L01 được dùng làm kênh điều khiển không dây giữa Controller (tay cầm) và Robot:

- Controller → Robot: Gửi lệnh di chuyển (FORWARD, BACKWARD, LEFT, RIGHT, STOP) và lệnh đặc biệt (RTH, SET_HOME, CLEAR_HOME)
- Robot → Controller: Send ACK confirmation (for SET_HOME, CLEAR_HOME command)

Ưu điểm so với Wi-Fi/Bluetooth:

- Tầm xa hơn (lên đến 1 km với PA/LNA)
- Độ trễ thấp hơn (1ms)
- Tiêu thụ điện ít hơn
- Không cần cấu hình SSID/Password

1.10 Cảm biến siêu âm HC-SR04

HC-SR04 là cảm biến đo khoảng cách bằng sóng siêu âm (ultrasonic).

1.10.1 Nguyên lý hoạt động

Nguyên lý:

1. Module phát xung siêu âm tần số 40 kHz
 2. Sóng siêu âm phản xạ lại khi gặp vật cản
 3. Module nhận sóng phản xạ và đo thời gian đi-về
 4. Tính khoảng cách: $distance = (timespeed_{of sound})/2$
- Tốc độ âm thanh trong không khí: 340 m/s = 29 μs/cm
Công thức: Khoảng cách (cm) = Thời gian (μs) / 29 / 2

1.10.2 Thông số kỹ thuật

- Dải đo: 2 cm – 400 cm
 - Độ phân giải: 0.3 cm
 - Góc đo: 15° (cone-shaped)
 - Điện áp: 5V DC
 - Dòng tiêu thụ: 15 mA
 - Tần số xung: 40 kHz
 - Thời gian đo: 50 ms (khuyến nghị ≥ 60 ms giữa các lần đo)
- Chân:
- VCC: 5V



Hình 1.12: HC-SR04

- GND: Ground
- TRIG: Input trigger (10 μ s HIGH pulse)
- ECHO: Output echo (HIGH duration = thời gian đi-về)

1.10.3 Lưu ý khi sử dụng

1. Góc phản xạ: Bề mặt vuông góc với cảm biến cho kết quả tốt nhất. Bề mặt nghiêng hoặc mềm (vải, xốp) có thể không phản xạ tốt.

2. Nhiệt độ: Tốc độ âm thanh thay đổi theo nhiệt độ. Ở 0°C là 331 m/s, ở 20°C là 343 m/s.

3. Multiple echoes: Trong môi trường có nhiều vật phản xạ, có thể nhận được nhiều echo.

4. Thời gian chờ: Đợi ít nhất 60 ms giữa các lần đo để tránh echo cũ.

5. Điện áp ECHO: HC-SR04 output 5V, ESP32 chỉ chịu 3.3V. Cần dùng bộ chia áp hoặc dùng module HC-SR04+ (3.3V compatible).

1.10.4 Ứng dụng trong dự án

Robot sử dụng 4 cảm biến HC-SR04 gắn ở 4 hướng:

- Front (Trước): Phát hiện vật cản phía trước
- Back (Sau): Phát hiện vật cản phía sau (khi lùi)
- Left (Trái): Phát hiện vật cản bên trái
- Right (Phải): Phát hiện vật cản bên phải

Chiến lược đọc Round-Robin (trong FreeRTOS):

Thay vì đọc cả 4 cảm biến cùng lúc (mất 200ms), TaskSensor đọc từng cảm biến một theo vòng tròn mỗi 20ms:

Cycle 0: Đọc Front (20ms)

Cycle 1: Đọc Back (20ms)

Cycle 2: Đọc Left (20ms)

Cycle 3: Đọc Right (20ms)

→ Lặp lại

Kết quả: Mỗi cảm biến được cập nhật mỗi 80ms, nhưng không block TaskControl.

1.11 Màn hình OLED SSD1306

1.11.1 Giới thiệu

SSD1306 là IC driver màn hình OLED (Organic Light-Emitting Diode) độ phân giải 128x64 hoặc 128x32 pixel. OLED có ưu điểm tương phản cao, góc nhìn rộng, không cần đèn nền, và tiêu thụ điện ít.

1.11.2 Thông số kỹ thuật

-Độ phân giải: 128x64 pixels (phổ biến nhất) hoặc 128x32

- Kích thước: 0.96 inch (đường chéo)
- Màu sắc: Đơn sắc (trắng, xanh dương, hoặc vàng-xanh)
- Giao tiếp: I2C hoặc SPI
- Địa chỉ I2C: 0x3C hoặc 0x3D (tùy module)
- Điện áp: 3.3V - 5V
- Dòng tiêu thụ: 20mA (tất cả pixel sáng)

1.11.3 Ứng dụng trong đề tài

Robot hiển thị 2 màn hình xoay vòng mỗi 5 giây:

Màn hình 1: Trạng thái robot

MODE: RTH (chữ lớn)

Head: 245.3 deg

Home: 21.02749, 105.83416

ToHome: 15.2m

F:45 L:120 R:80 B:200

Màn hình 2: Thông tin GPS

GPS INFO (chữ lớn)

Lat: 21.0275

Lon: 105.8342

Sats: 8 | Age: 250ms
MAG_CALIB : OK

1.12 Module joystick PS2

1.12.1 Giới thiệu

Module Joystick đơn thường được sử dụng để làm tay cầm hoặc cần điều khiển, mạch có kích thước nhỏ gọn với bốn lỗ ốc trên mạch rất dễ gá bắt. Module joystick điều khiển đơn rất dễ sử dụng, chỉ cần cấp nguồn cho mạch, các tín hiệu trả ra bao gồm tín hiệu Analog của hai trục X,Y và 1 nút bấm.

Arduino PS2 JoyStick hoạt động giống như nút nhấn trên bàn phím trên tay game Playstation. Bên trong module là 2 biến trở 10k.

Khi xoay tay cầm trên module thì sẽ thay đổi điện trở thông qua module Analog ta sẽ biết được giá trị của trục X và trục Y, đồng thời trên module tích hợp thêm 1 nút nhấn để nhận biết khi nhấn nút xuống.

1.12.2 Thông số kỹ thuật

Nguồn cấp: Tùy chọn, thường cấp 3.3 hoặc 5VDC.

Kiểu dạng tín hiệu ngõ ra 1 Digital và 2 Analog (1 nút nhấn và hai trục X, Y), mức tín hiệu theo nguồn cấp vào.

Kích thước: 4.0cm x 2.6cm x 3.2cm

Trọng lượng: 12g

1.13 Module QMC5883L

1.13.1 Giới thiệu

QMC5883L là một IC cảm biến từ trường (magnetometer) 3 trục được sản xuất bởi QST Corporation. Module GY-271 tích hợp chip QMC5883L và được sử dụng làm la bàn điện tử với độ chính xác cao.

Tại sao dùng QMC5883L riêng thay vì magnetometer tích hợp trong MPU9250?

- Độ nhạy cao hơn: 12 LSB/Gauss (vs 0.6 μ T/LSB của MPU9250)
- Ít nhiễu hơn: IC riêng biệt, không bị ảnh hưởng bởi accel/gyro
- Tần số cập nhật cao hơn: Lên đến 200 Hz
- Calibration dễ dàng hơn với thư viện QMC5883LCompass

1.13.2 Thông số kỹ thuật

- Điện áp hoạt động: 3 ~ 5V
- Chuẩn giao tiếp I2C, địa chỉ 0x0D
- Dải đo: 1.3-8 gauss
- PCB Board Size: 1.5 x 1.3cm/0.59"x 0.51"(L*W)

1.13.3 Nguyên lí đo từ trường

QMC5883L đo từ trường trái đất (geomagnetism) theo 3 trục X, Y, Z:

- Trục X: Hướng về phía trước robot
- Trục Y: Hướng sang trái robot
- Trục Z: Hướng xuống dưới (vuông góc với mặt đất)

Từ trường trái đất:

- Cường độ: 25-65 μT (tùy vị trí địa lý)
- Hà Nội: 45 μT
- Góc nghiêng (Inclination): 45° (phần Bắc bán cầu)

Tính toán góc Heading (Yaw):

$\text{heading} = \text{atan2}(\text{magY}, \text{magX}) * 180 / \text{PI}$

Tuy nhiên, công thức này chỉ đúng khi robot nằm hoàn toàn ngang. Khi robot bị nghiêng, cần bù nghiêng.

1.13.4 Bù nghiêng

Vấn đề: Khi robot đi trên địa hình không bằng phẳng, trục Z của từ kế bị nghiêng $\rightarrow$ đọc sai từ trường $\rightarrow$ heading sai.

Giải pháp: Sử dụng góc Roll và Pitch từ MPU9250 để hiệu chỉnh:

```
1 // B c 1: c g c nghi ng t MPU9250
2 if (mpu.accelUpdate() == 0) {
3     float ax = mpu.accelX();
4     float ay = mpu.accelY();
5     float az = mpu.accelZ();
6
7     float roll = atan2(ay, az);
8     float pitch = atan2(-ax, sqrt(ay*ay + az*az));
9
10    // B c 2: c t k QMC5883L ( t r offset v
11        nh n scale)
12    qmc.read();
13    float mx = ((float)qmc.getX() - magX_off) * magScaleX;
14    float my = ((float)qmc.getY() - magY_off) * magScaleY;
15    float mz = ((float)qmc.getZ() - magZ_off) * magScaleZ;
16
17    // B c 3: B nghi ng (Tilt Compensation)
18    float magX_comp = mx * cos(pitch) - mz * sin(pitch);
19    float magY_comp = mx * sin(roll) * sin(pitch) +
20        my * cos(roll) -
21        mz * sin(roll) * cos(pitch);
22
23    // B c 4: T nh heading
24    float heading = atan2(magY_comp, magX_comp) * 180.0 / PI;
25    if (heading < 0) heading += 360.0;
26 }
```

1.13.5 Calibration (Hiệu chỉnh)

Từ kế rất nhạy cảm với nhiễu từ (motor, pin, dây điện). Bắt buộc phải calibration trước khi sử dụng!

Hai loại nhiễu:

1. Hard-iron distortion (Nhiều cứng):

- Nguồn: Từ trường cố định từ motor, pin, PCB
- Hiệu ứng: Dịch tâm (offset)
- Giải pháp: Trừ offset

2. Soft-iron distortion (Nhiều mềm):

- Nguồn: Vật liệu từ tính mềm làm méo từ trường
- Hiệu ứng: Biến dạng hình cầu thành ellipse
- Giải pháp: Nhân scale factor

Thuật toán calibration trong dự án:

```
1 void runMagCalibration(unsigned long durationMs = 20000) {
2   float minX = FLT_MAX, minY = FLT_MAX, minZ = FLT_MAX;
3   float maxX = -FLT_MAX, maxY = -FLT_MAX, maxZ = -FLT_MAX;
4
5   // Thu thập dữ liệu trong 20 giây (quay robot 360 độ nhiều lần)
6   unsigned long start = millis();
7   while (millis() - start < durationMs) {
8     qmc.read();
9     float mx = (float)qmc.getX();
10    float my = (float)qmc.getY();
11    float mz = (float)qmc.getZ();
12
13    // Tìm min/max mới
14    if (mx < minX) minX = mx;
15    if (mx > maxX) maxX = mx;
16    // ... tìm giá trị Y, Z
17    delay(20);
18  }
19
20  // Tính offset (hard-iron compensation)
21  float offX = (maxX + minX) / 2.0f;
22  float offY = (maxY + minY) / 2.0f;
23  float offZ = (maxZ + minZ) / 2.0f;
24
25  // Tính scale (soft-iron compensation)
26  float rangeX = (maxX - minX) / 2.0f;
27  float rangeY = (maxY - minY) / 2.0f;
28  float rangeZ = (maxZ - minZ) / 2.0f;
29  float avgRange = (rangeX + rangeY + rangeZ) / 3.0f;
30
31  float scaleX = avgRange / rangeX;
32  float scaleY = avgRange / rangeY;
33  float scaleZ = avgRange / rangeZ;
34
35  // Lưu vào NVS (Preferences)
36  saveMagCalibrationToPrefs(offX, offY, offZ, scaleX, scaleY,
    scaleZ);
```

Sử dụng calibration:

```

1 qmc.read();
2 float mx = ((float)qmc.getX() - magX_off) * magScaleX;
3 float my = ((float)qmc.getY() - magY_off) * magScaleY;
4 float mz = ((float)qmc.getZ() - magZ_off) * magScaleZ;

```

1.13.6 Ưu và nhược điểm

Ưu điểm: - Độ chính xác cao (1-2° sau calibration)

- Tiêu thụ điện thấp (75 μ A)
- Tần số cập nhật cao (200 Hz)
- Giá khá rẻ, phổ biến (50,000 VNĐ)
- Thư viện Arduino dễ sử dụng

Nhược điểm: - Rất nhạy cảm với nhiễu từ $\rightarrow$ BẮT BUỘC calibration

- Cần re-calibration thường xuyên (mỗi 1-2 tháng hoặc khi thay đổi phần cứng)
- Không hoạt động gần vật liệu từ tính (loa, nam châm)
- Bị ảnh hưởng bởi cột điện, đường ray tàu điện

Mạch hạ áp LM2596

Mạch hạ áp LM2596 là một bộ chuyển đổi DC–DC dạng **buck (step-down)** dùng để giảm điện áp đầu vào xuống một mức điện áp thấp hơn với hiệu suất cao (khoảng 80–90%). LM2596 hoạt động ở tần số switching 150 kHz và cung cấp dòng tải tối đa khoảng 3 A, vì vậy nó thường được dùng trong các module hạ áp 12V xuống 5V, 9V, 3.3V hoặc loại điều chỉnh được (ADJ).

Một mạch LM2596 cơ bản gồm các linh kiện:

- IC LM2596 (phiên bản cố định hoặc adjustable)
- Cuộn cảm giá trị khoảng 33–47 μ H
- Tụ lọc đầu vào (100–470 μ F)
- Tụ lọc đầu ra (220–470 μ F)
- Đi-ốt Schottky (ví dụ SS34)
- Điện trở chia áp (chỉ với bản adjustable) để tạo điện áp hồi tiếp FB

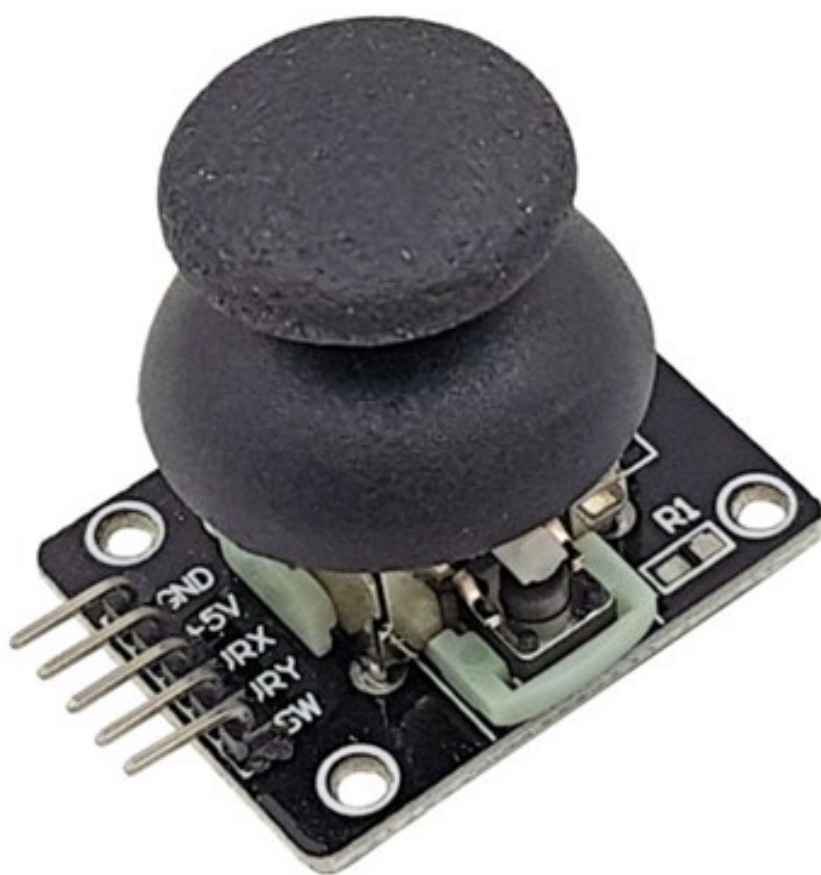
Nguyên lý hoạt động: LM2596 đóng/ngắt transistor nội, tạo xung tại chân SW. Tín hiệu này đi qua cuộn cảm và tụ đầu ra để được lọc thành điện áp một chiều ổn định. Mạch hồi tiếp (Feedback) sẽ điều chỉnh độ rộng xung để giữ điện áp ra đúng giá trị đặt.

Ưu điểm chính:

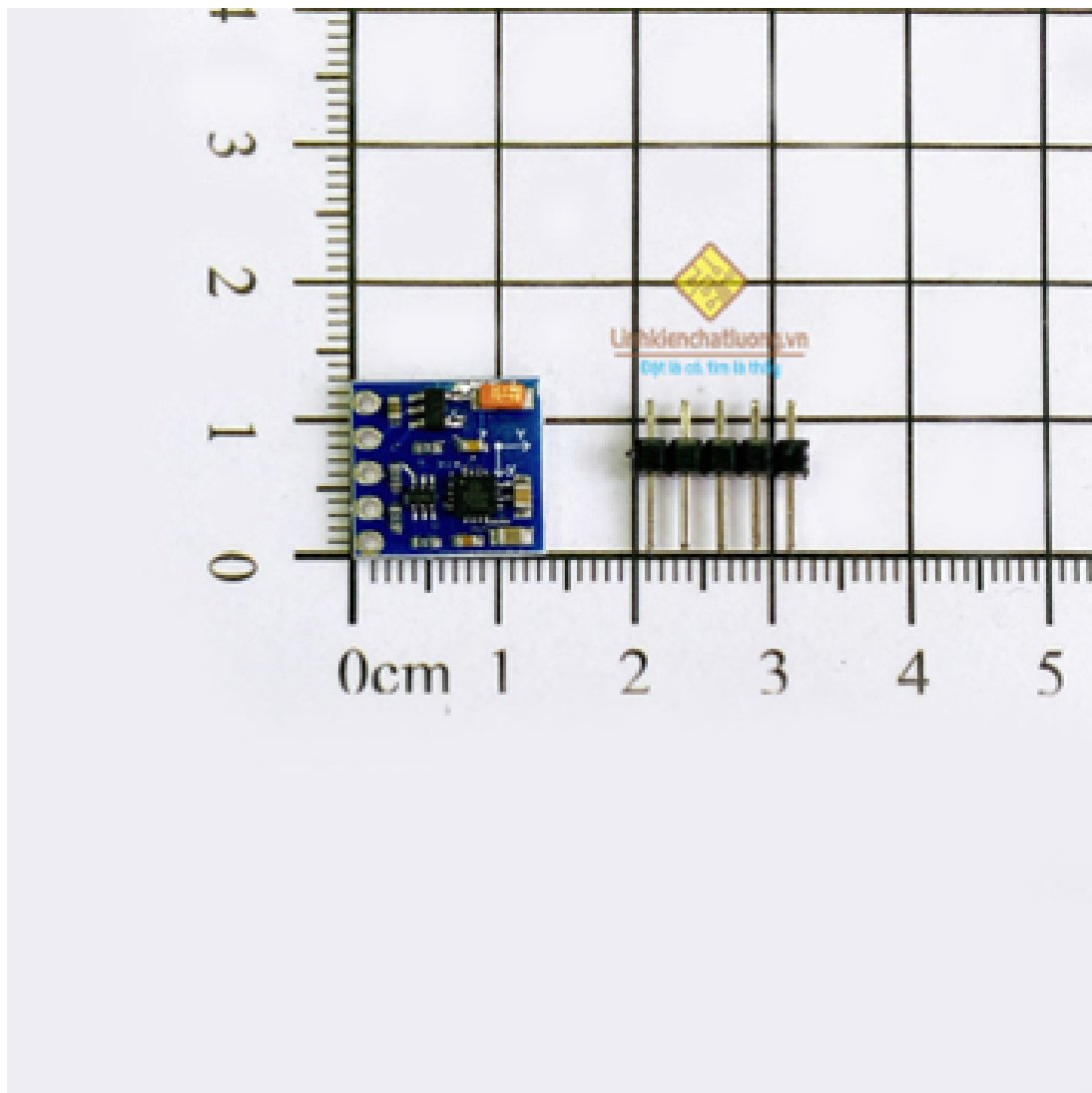
- Hiệu suất cao hơn nhiều so với ổn áp tuyến tính (7805)
- Dòng tải lớn (tới 3 A)
- Hoạt động ổn định, dễ sử dụng



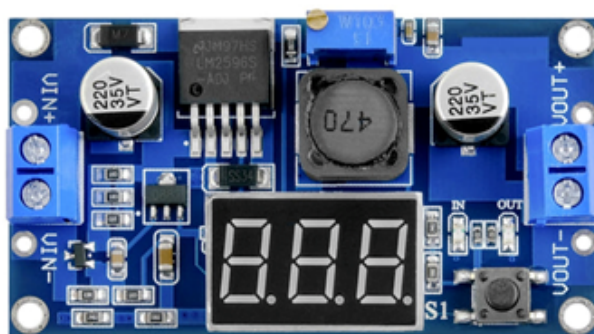
Hình 1.13: Màn hình OLED SSD1306



Hình 1.14: Module joystick PS2



Hình 1.15: Module QMC5883L



Hình 1.16: Mạch hạ áp LM2596

Chương 2

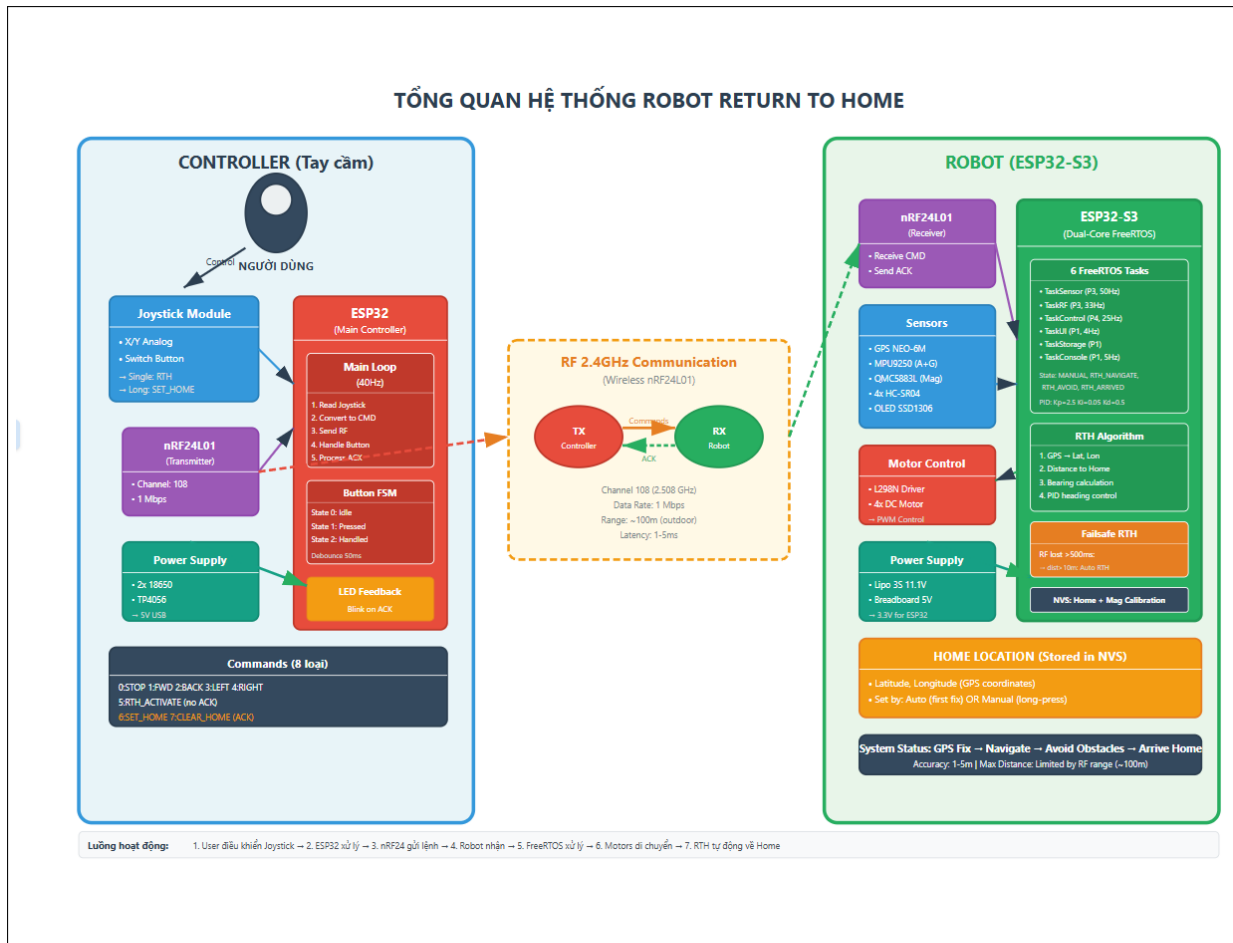
THIẾT KẾ HỆ THỐNG

2.1 Mô tả hoạt động của hệ thống

2.1.1 Tổng quan

Hệ thống robot Return To Home (RTH) bao gồm hai phần chính:

- **Controller (tay cầm điều khiển):** Sử dụng ESP32 với joystick analog để gửi lệnh điều khiển đến robot qua sóng RF 2.4GHz (nRF24L01).
- **Robot (ESP32-S3):** Nhận lệnh từ controller, thực hiện di chuyển theo lệnh thủ công hoặc tự động quay về điểm xuất phát (Home) dựa trên GPS và la bàn điện tử.



Hình 2.1: Tổng quan hệ thống

2.1.2 Chức năng chính

A. Điều khiển thủ công (MANUAL mode)

Người dùng điều khiển robot bằng joystick với các thao tác:

- Đẩy lên: tiến
- Đẩy xuống: lùi
- Đẩy trái: quay trái
- Đẩy phải: quay phải
- Thả ra: dừng

B. Lưu vị trí Home

1. Người dùng nhấn giữ nút joystick.
2. Controller gửi lệnh SET_HOME.
3. Robot lưu tọa độ GPS hiện tại vào NVS.
4. Robot gửi ACK về controller.

C. Return To Home (RTH mode)

1. Người dùng nhấn nhanh (single-click) nút joystick
2. Controller gửi lệnh $RTH_ACTIVATE$.
3. Robot kiểm tra trạng thái Home, sau đó nếu đã có Home thì bắt đầu điều hướng tự động ($RTH_NAVIGATE$).

D. Điều hướng tự động (RTH_NAVIGATE)

Robot thực hiện:

1. Đọc vị trí hiện tại từ GPS (latitude, longitude)
2. Tính khoảng cách đến Home bằng công thức Haversine
3. Tính góc bearing cần đi đến Home (0-360°)
4. Đọc góc heading hiện tại từ MPU9250 (la bàn điện tử có bù nghiêng)
5. Tính sai lệch góc ($bearingError = bearing - heading$)
6. PID Controller điều chỉnh tốc độ 2 bánh để robot quay đúng hướng
 $rightSpeed = baseSpeed + PID_{output}$
 $leftSpeed = baseSpeed - PID_{output}$
6. Giảm tốc khi gần Home (<3m)
7. Dừng lại khi đến Home (<1m) → Chuyển sang $RTH_ARRIVED$.

E. Tránh vật cản (RTH_AVOID)

- Trong khi điều hướng về Home, nếu cảm biến trước phát hiện vật cản (<30cm):

1. Chuyển sang chế độ RTH_AVOID
1. Lùi 500 ms.
2. Quay trái/phải tùy không gian.
3. Quay lại $RTH_NAVIGATE$.

F. Xóa Home

- Người dùng nhấn đúp (double-click) nút joystick
- Controller gửi lệnh $CLEAR_HOME$
- Robot xóa tọa độ Home trong NVS
- Robot gửi ACK xác nhận

2.2 Kiến trúc FreeRTOS

Robot sử dụng 6 task chạy song song:

Task	Chức năng	Priority	Core	Tần số
TaskSensor	Đọc 4 siêu âm, GPS, MPU9250	3	0	50 Hz
TaskRF	Nhận lệnh từ nRF24L01	3	1	33 Hz
TaskControl	Máy trạng thái, PID, điều khiển động cơ	4	1	25 Hz
TaskUI	Cập nhật màn hình OLED	1	0	4 Hz
TaskStorage	Ghi NVS (không block task khác)	1	0	Tùy theo yêu cầu
TaskConsole	Xử lý lệnh Serial (Magnetometer calibration)	1	0	5 Hz

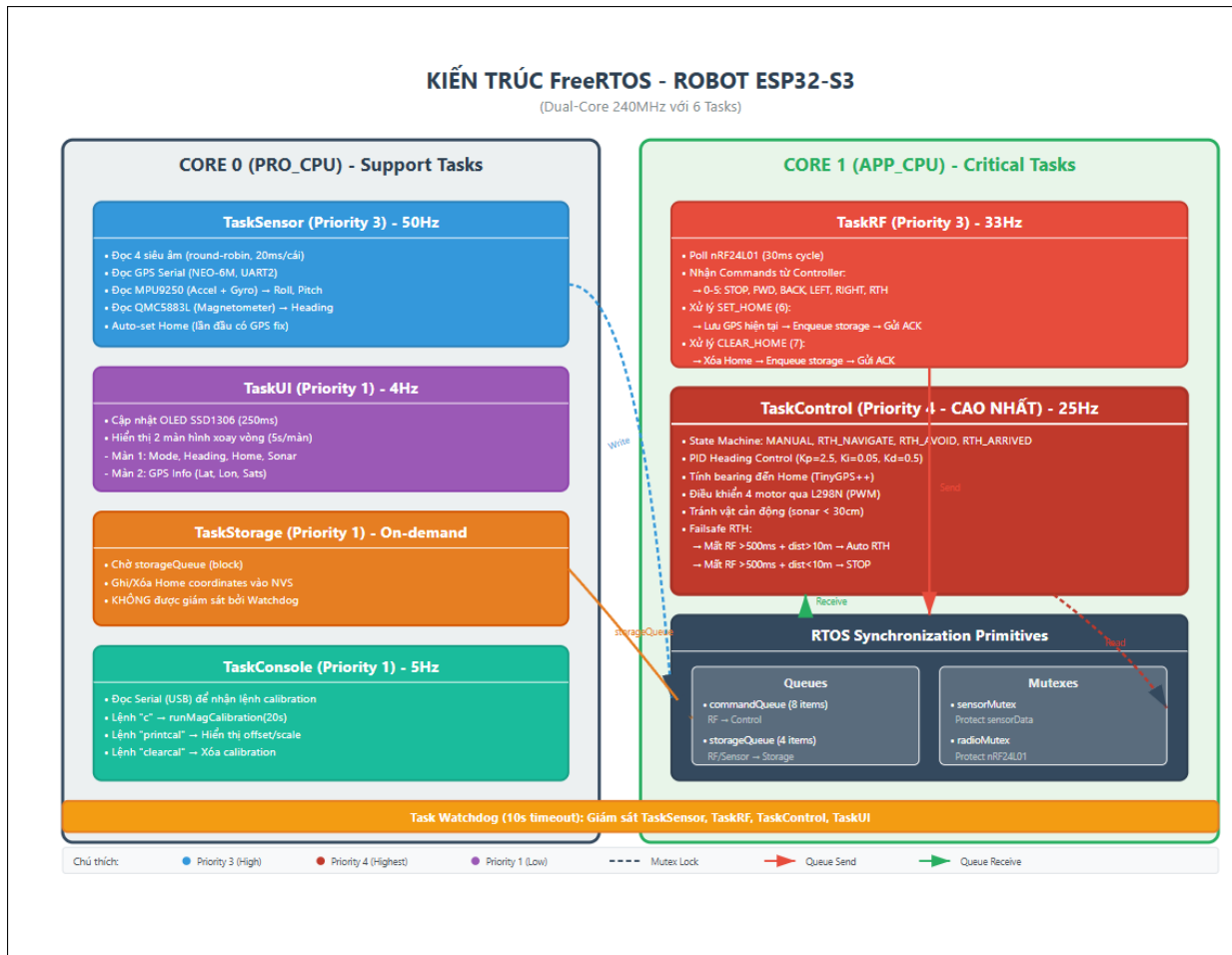
Bảng 2.1: Bảng mô tả các task trong hệ thống FreeRTOS

Đồng bộ hóa:

- sensorMutex: Bảo vệ dữ liệu cảm biến (GPS, heading, sonar)
- radioMutex: Bảo vệ truy cập nRF24L01
- commandQueue: Truyền lệnh từ TaskRF → TaskControl
- storageQueue: Truyền yêu cầu lưu Home từ TaskSensor → TaskStorage

Task Watchdog:

- Giám sát 4 task quan trọng (Sensor, RF, Control, UI)
- Timeout: 10 giây
- Nếu task nào không feed watchdog → ESP32 tự động reset
- TaskStorage và TaskConsole KHÔNG được giám sát (vì có thể block lâu)



Hình 2.2: Kiến trúc FreeRTOS của robot

2.3 Hệ thống tay cầm điều khiển (Controller)

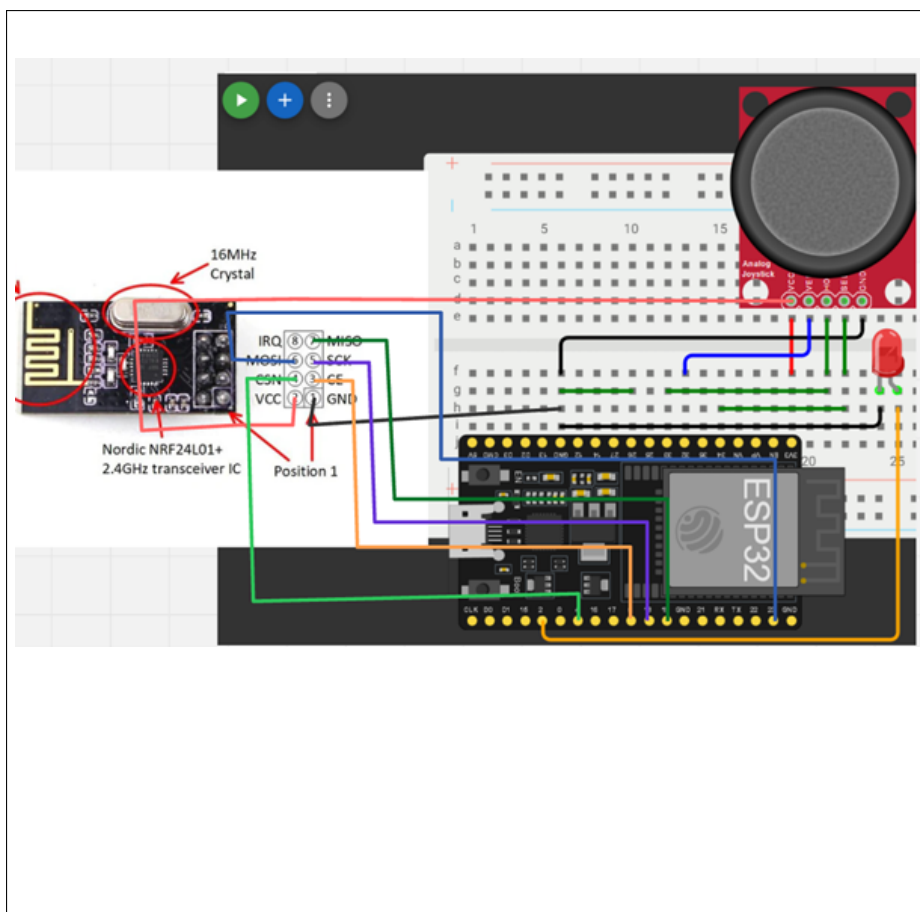
2.3.1 Tổng quan

Controller là thiết bị cầm tay cho phép người dùng điều khiển robot từ xa thông qua sóng RF 2.4GHz. Controller được xây dựng trên nền tảng ESP32 với các thành phần chính:

- Joystick 2 trục
- nRF24L01
- Pin 18650 và module sạc TP4056
- LED báo trạng thái

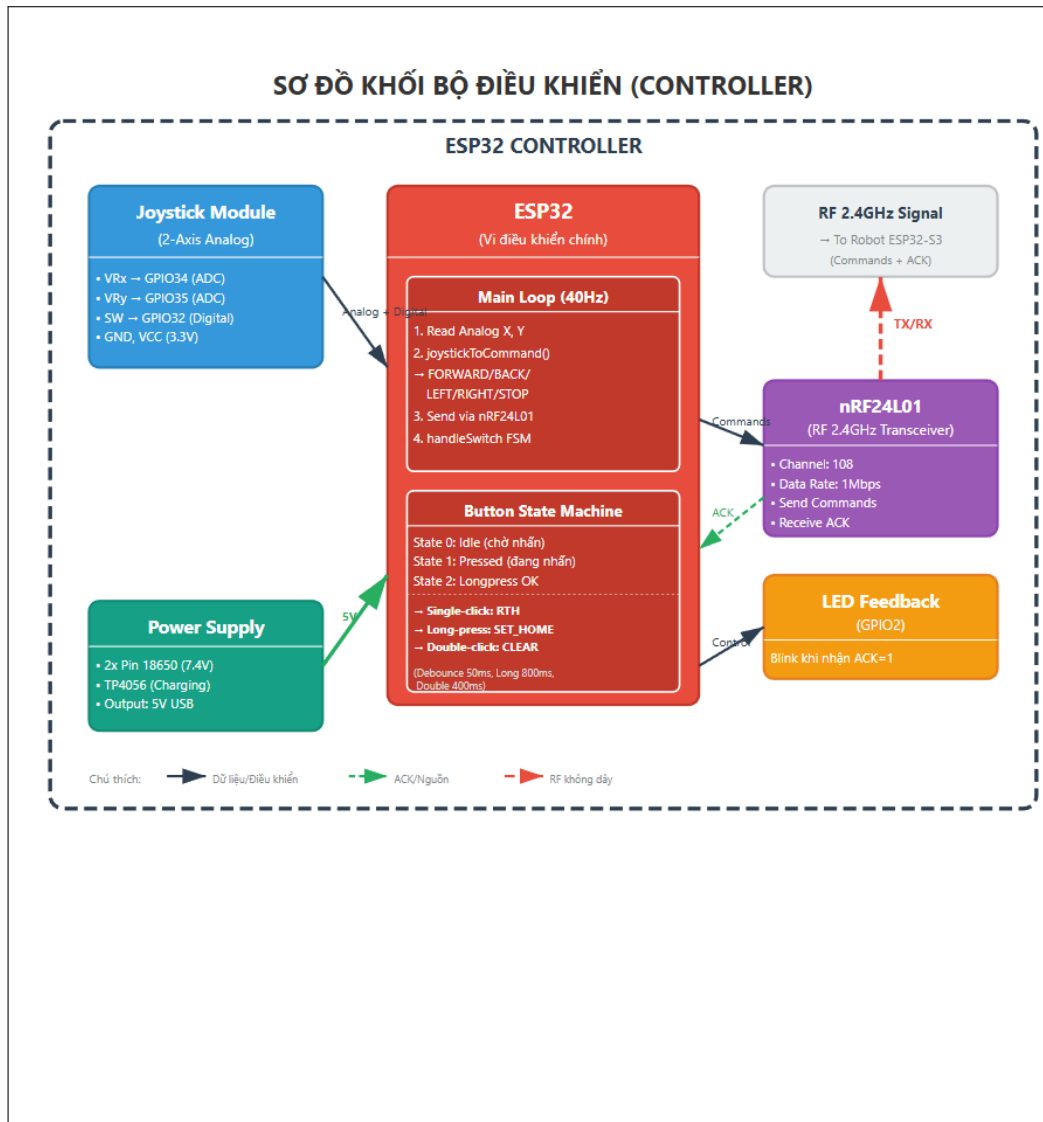
Chức năng:

- Điều khiển di chuyển: Joystick analog (X, Y) → Lệnh FORWARD, BACKWARD, LEFT, RIGHT, STOP
- Điều khiển đặc biệt: Nút nhấn joystick (SW) → RTH, SET_HOME, CLEAR_HOME
- Phản hồi trực quan: LED nhấp nháy khi nhận ACK thành công
- Truyền không dây: Gửi lệnh qua nRF24L01 với/không có xác nhận (ACK)



Hình 2.3: Hệ thống controller ESP32

2.3.2 Sơ đồ khối



Hình 2.4: Sơ đồ khối tay cầm điều khiển

2.3.3 Mô tả hoạt động chi tiết

A. Đọc và xử lý Joystick

Bước 1: Đọc giá trị analog

```
1 int xValue = analogRead(JOY_X);  
2 int yValue = analogRead(JOY_Y);
```

Bước 2: Phân vùng và chuyển đổi thành lệnh

```
1 Command joystickToCommand(int xValue, int yValue) {  
2   // u t i n Y ( t i n / l i ) t r c X (quay)  
3   if (yValue < 1548) { // y l n m nh  
4     return CMD_FORWARD;  
5   } else if (yValue > 2548) { // y x u ng m nh  
6     return CMD_BACKWARD;  
7   }
```

```

7   } else if (xValue < 1548) { // y tr i m nh
8       return CMD_LEFT;
9   } else if (xValue > 2548) { // y p h i m nh
10      return CMD_RIGHT;
11  } else { // Trong deadzone
12      return CMD_STOP;
13  }
14 }

```

Bước 3: Gửi lệnh qua nRF24L01 (chỉ khi thay đổi)

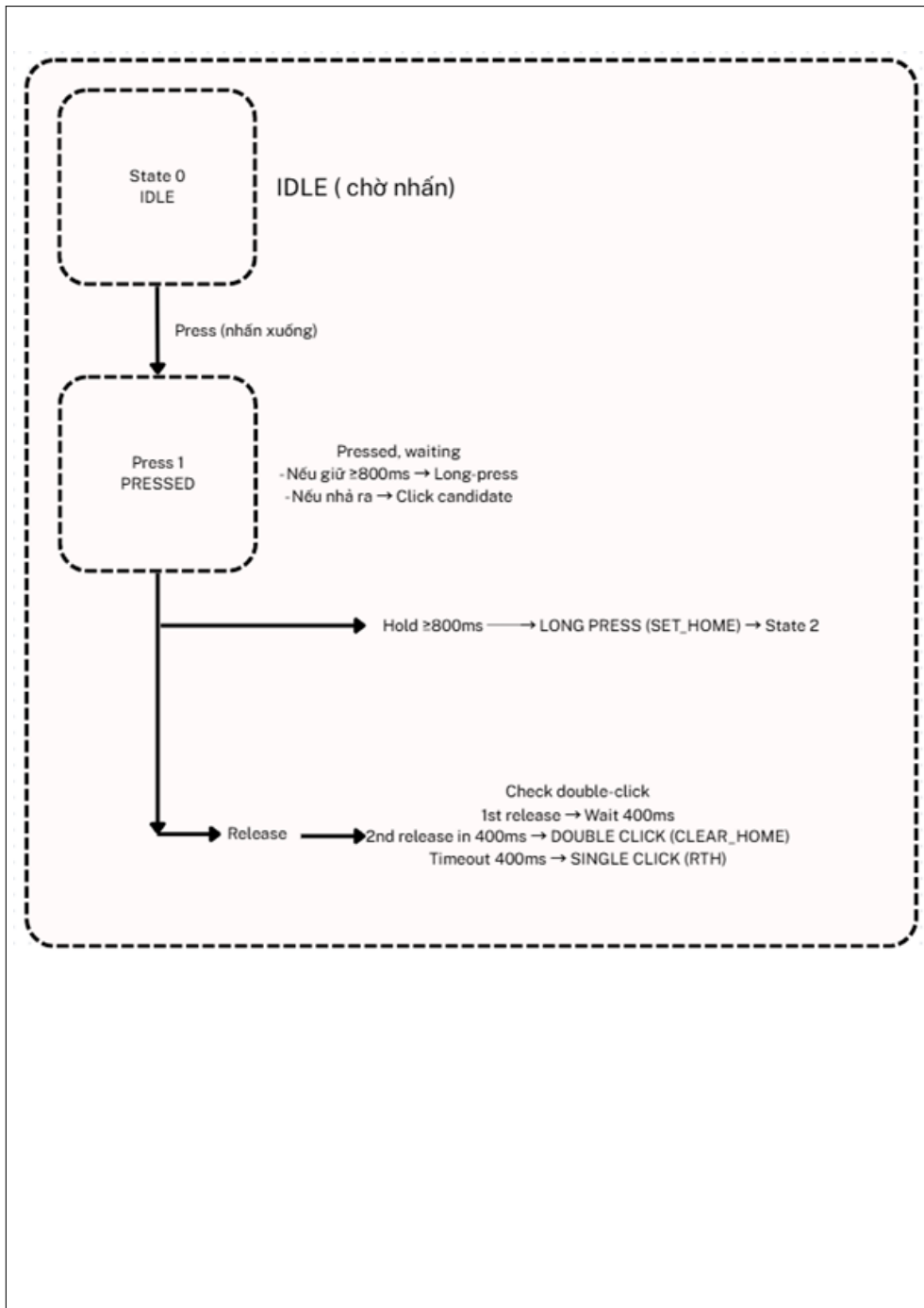
```

1  if (cmd != previousCommand) {
2      sendCommandNoAck(cmd); // Kh ng c n ACK cho l nh di
3      c h u y n
4      previousCommand = cmd;
5  }

```

B. Xử lý nút nhấn (Button FSM)

Controller sử dụng Finite State Machine để phân biệt 3 loại sự kiện nhấn nút: Sơ đồ trạng thái:



Hình 2.5: Finite State Machine xử lý nút joystick

Code FSM:

```

1 void handleClick() {
2   if (!rthActive) {
3     Serial.println("BTN: Activate RTH");
4     sendCommand(RTH_ACTIVATE);
5     rthActive = true;
6   } else {
7     Serial.println("BTN: Cancel RTH");
8     sendCommand(STOP);
9     rthActive = false;
  
```

```

10 }
11 }
12
13 void handleDoubleClick() {
14     Serial.println("BTN: Clear Home");
15     sendCommand(CLEAR_HOME);
16     // Nh y 3 l n
17     for(int i=0; i<3; i++) { digitalWrite(PIN_LED, HIGH); delay(50);
18         digitalWrite(PIN_LED, LOW); delay(50); }
19 }
20
21 void handleLongPress() {
22     Serial.println("BTN: Set Home");
23     sendCommand(SET_HOME);
24     // S ng d i
25     digitalWrite(PIN_LED, HIGH); delay(500); digitalWrite(PIN_LED,
26         LOW);
27 }
28
29 // --- H M M I : X L N H N 3 L N ---
30 void handleMultiClick() {
31     if (btn.getNumberClicks() == 3) {
32         Serial.println("BTN: SEND CALIBRATE MAG (CMD 8)");
33         sendCommand(CALIBRATE_MAG);
34
35         // Nh y n LED li n t c c nh b o
36         for(int i=0; i<10; i++) {
37             digitalWrite(PIN_LED, HIGH); delay(50);
38             digitalWrite(PIN_LED, LOW); delay(50);
39         }
40     }
41 }

```

Timing:

- Debounce: 50ms (lọc nhiễu cơ học)
- Long-press: 800ms (nhấn giữ)
- Double-click window: 400ms (khoảng cách 2 lần nhấn)

C. Gửi lệnh qua nRF24L01

Hai chế độ gửi:

1. Không có ACK (cho lệnh di chuyển):

```

1 bool sendCommandNoAck(Command cmd) {
2     uint8_t payload = (uint8_t)cmd;
3     radio.stopListening();           // TX mode
4     bool ok = radio.write(&payload, 1);
5     radio.startListening();          // RX mode ( s n s ng n h n ACK)
6     return ok;
7 }

```

Đặc điểm:

- Nhanh (1-2ms)
- Không chờ xác nhận
- Phù hợp cho lệnh realtime (FORWARD, STOP...)

2. Có ACK (cho lệnh quan trọng):

```

1 bool sendCommandWithAck(Command cmd, timeout=500ms, retries=2) {
2   for (int attempt = 0; attempt <= retries; ++attempt) {
3     radio.stopListening();
4     radio.write(&cmd, 1);          // G i   l   nh
5     radio.startListening();        // C h   ACK
6
7     unsigned long start = millis();
8     while (millis() - start < timeout) {
9       if (radio.available()) {
10        uint8_t ack;
11        radio.read(&ack, 1);
12        return (ack == 1);          // ACK=1       t h   n h   c   n g
13      }
14      delay(5);
15    }
16    // Timeout       retry
17  }
18  return false; // T h   t       b   i   sau 3   l   n   t   h
19 }

```

Đặc điểm:

- Chậm hơn (50-500ms)
- Đảm bảo lệnh được thực hiện
- Có cơ chế retry (3 lần)
- Phù hợp cho SET_HOME, CLEAR_HOME

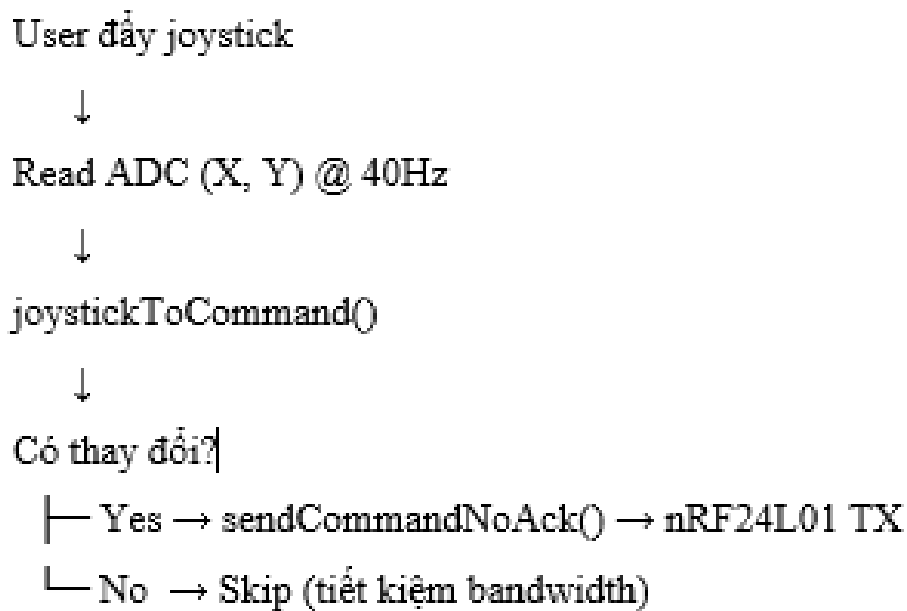
D. Phản hồi LED

LED nhấp nháy khi:

- Nhận ACK=1 từ robot (SET_HOME hoặc CLEAR_HOME thành công)
- Không nhấp nháy nếu timeout hoặc ACK=0

2.3.4 Các cơ chế hoạt động

Chế độ 1: Điều khiển thủ công (Manual Control)



Hình 2.6: Luồng hoạt động chế độ điều khiển thủ công

Luồng hoạt động Quy trình xử lý:

1. User đẩy joystick.

2. Bộ điều khiển đọc ADC (X, Y) ở tần số 40 Hz.
3. Hàm `joystickToCommand()` chuyển đổi giá trị ADC thành lệnh.
4. Nếu lệnh thay đổi: gửi `sendCommandNoAck()` qua nRF24L01.
5. Nếu không thay đổi: bỏ qua để tiết kiệm băng thông.

Đặc điểm:

- Điều khiển mượt (40 Hz).
- Không dùng ACK $\rightarrow$ độ trễ thấp (< 5 ms).
- Chỉ gửi khi lệnh thay đổi.

Chế độ 2: Kích hoạt RTH (Single-click)

```
Controller: Single-click joystick
      ↓
Send CMD_RTH (no ACK)
      ↓
Robot: TaskRF receives → Queue → TaskControl
      ↓
TaskControl: Check homeValid
      └─ If false → Refuse, stay MANUAL
      └─ If true → Switch to RTH_NAVIGATE|
```

Hình 2.7: Luồng hoạt động chế độ Single-click

Luồng hoạt động Quy trình:

1. User nhấn nhanh nút joystick.
2. FSM chuyển State 0 → State 1 → State 0.
3. Nhả trước 800 ms và không có click thứ hai trong 400 ms.

4. Hệ thống nhận dạng **Single Click**.
5. Gửi `sendCommandNoAck(CMD_RTH)`.
6. Robot chuyển sang chế độ `RTH_NAVIGATE`.

Đặc điểm:

- Không cần ACK.
- Robot tự kiểm tra `homeValid`.
- Phản hồi nhanh, không block.

Chế độ 3: Lưu Home (Long-press)

```
Controller: Long-press joystick
↓
Send CMD_SET_HOME (with ACK)
↓
Robot: TaskRF receives → Queue → TaskControl
↓
TaskControl: Read current GPS → Set homeLat, homeLon
↓
TaskSensor: Enqueue to StorageQueue
↓
TaskStorage: Write to NVS (Preferences)
↓
Send ACK = 1 back to Controller
↓
Controller: LED blink (success)
```

Hình 2.8: Luồng hoạt động chế độ Long-press

Luồng hoạt động Quy trình:

1. User nhấn giữ nút joystick.

2. FSM chuyển State 0 $\rightarrow$ State 1.
3. Giữ ≥ 800 ms $\rightarrow$ nhận dạng **Long Press**.
4. Gửi:

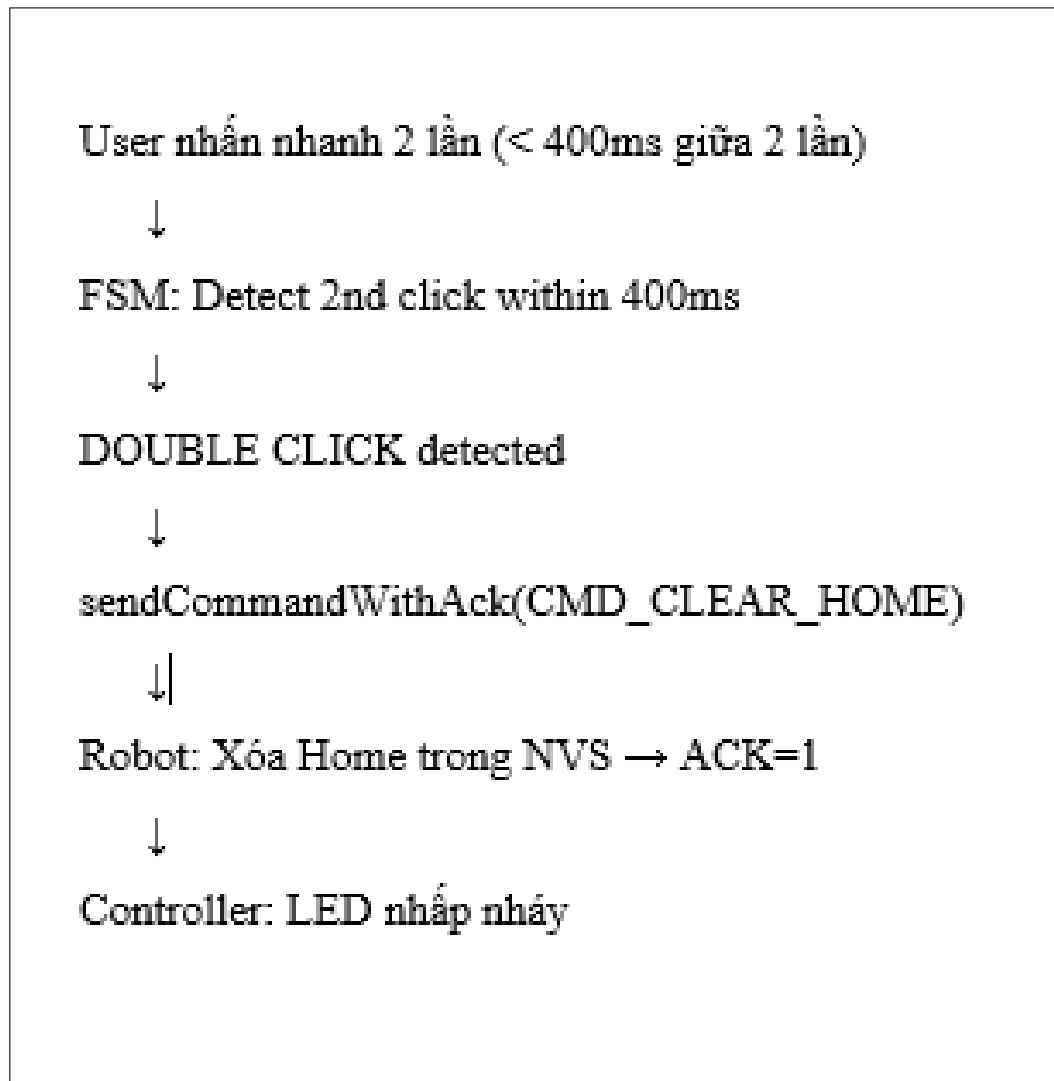
```
sendCommandWithAck(CMD_SET_HOME, timeout = 500ms, retry = 2)
```

5. Robot lưu GPS $\rightarrow$ ghi NVS $\rightarrow$ gửi ACK = 1.
6. Controller nhận ACK $\rightarrow$ LED nhấp nháy 80 ms.
7. FSM chuyển State 2 $\rightarrow$ chờ nhả $\rightarrow$ State 0.

Đặc điểm:

- Có ACK $\rightarrow$ đảm bảo lưu Home thành công.
- Retry tối đa 3 lần.
- LED báo trực quan.

Chế độ 4: Xóa Home (Double-click)



Hình 2.9: Luồng hoạt động chế độ Double-click

Luồng hoạt động Quy trình:

1. User nhấn nhanh 2 lần (≤ 400 ms giữa hai click).
2. FSM phát hiện click thứ hai trong khoảng thời gian hợp lệ.
3. Nhận dạng **Double Click**.
4. Gửi `sendCommandWithAck(CMD_CLEAR_HOME)`.
5. Robot xóa Home trong NVS $\rightarrow$ gửi $ACK = 1$.
6. Controller nhận $ACK \rightarrow$ LED nhấp nháy báo thành công.

Chế độ	Trigger	Lệnh gửi	Có ACK?	Thời gian	LED?
Manual Control	Joystick X/Y	FORWARD, BACK, LEFT, RIGHT, STOP	Không	< 5ms	Không
RTH Activate	Single-click	CMD_RTH	Không	< 5ms	Không
Set Home	Long-press (> 800ms)	CMD_SET_HOME	Có	50 – 500ms	Có
Clear Home	Double-click (< 400ms)	CMD_CLEAR_HOME	Có	50 – 500ms	Có

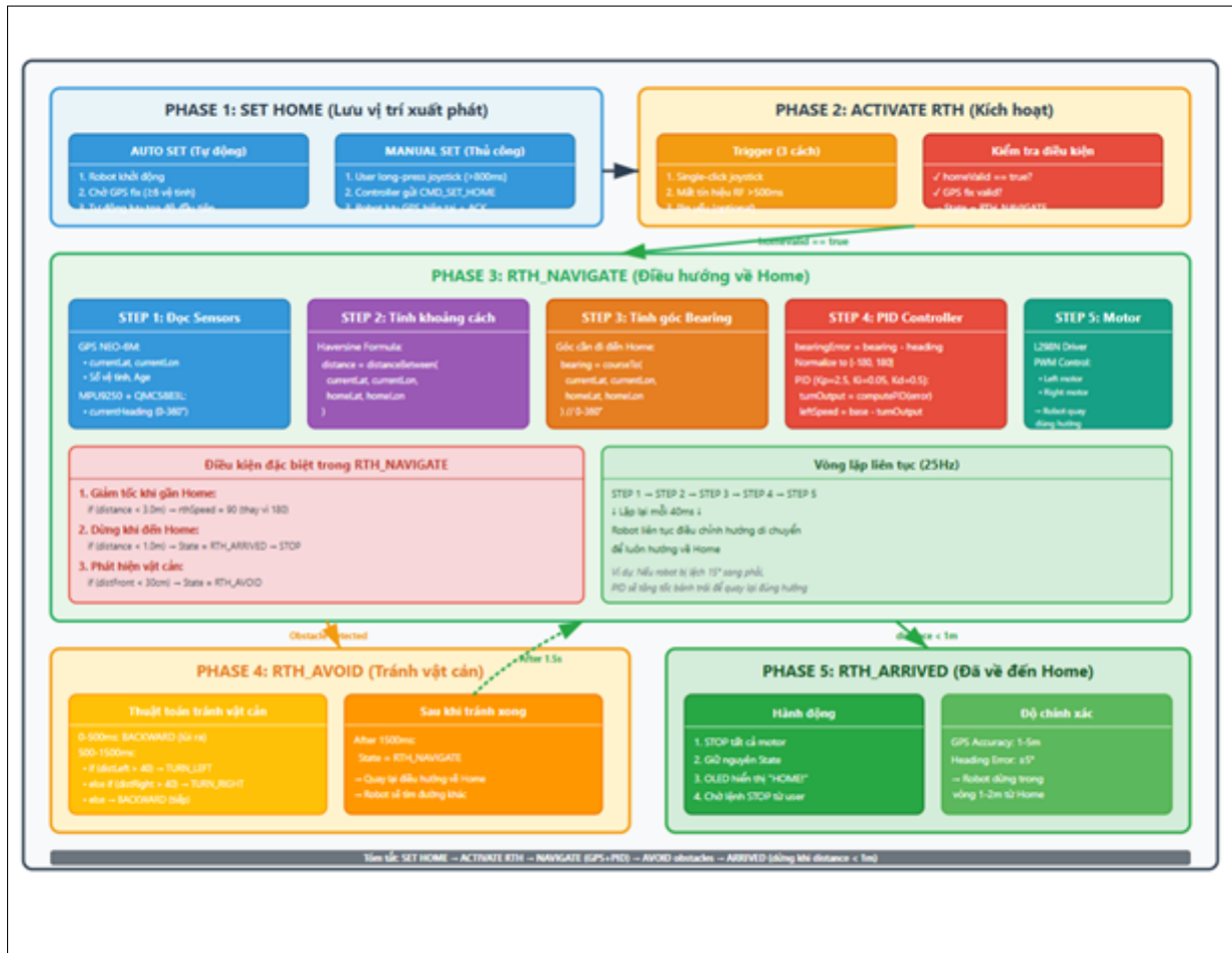
Bảng 2.2: Bảng tóm tắt các chế độ điều khiển

2.3.5 Bảng so sánh các chế độ

2.3.6 Xử lý lỗi và failsafe

1. **Mất kết nối RF:** - Controller không có failsafe (không biết robot có nhận không)
 - Robot có failsafe: Dừng lại nếu không nhận lệnh trong 500ms
2. **ACK timeout:** - Retry 3 lần (mỗi lần 500ms)
 - Nếu thất bại → không blink LED (user biết lệnh không thành công)
3. **Nhiều ADC:**
 - Deadzone ± 500 lọc nhiễu tự nhiên
 - Chỉ gửi khi command thay đổi (hysteresis)
4. **Bounce nút nhấn:**
 - Debounce 50ms lọc nhiễu cơ học
 - FSM đảm bảo không trigger nhầm

2.4 Sơ đồ khối hệ thống



Hình 2.10: Sơ đồ tổng thể hệ thống RTH

2.5 Luồng dữ liệu Return To Home

Loop 25Hz:

- Read GPS: currentLat, currentLon
- Compute distance = Haversine(current, home)
- If distance < 1m → RTH_ARRIVED (STOP)
- Compute bearing = courseTo(current, home)
- Read heading from MPU9250 (tilt-compensated)
- bearingError = bearing - heading
- Normalize bearingError to [-180, 180]
- PID.setpoint = 0, PID.input = bearingError
- turnOutput = computePID()
- leftSpeed = baseSpeed - turnOutput
- rightSpeed = baseSpeed + turnOutput
- If distFront < 30cm → RTH_AVOID
- controlMotorsPWM(leftSpeed, rightSpeed)

Hình 2.11: Luồng dữ liệu Return To Home

2.6 Thuật toán Return To Home

2.6.1 Tổng quan

Robot sử dụng phương pháp GPS-based Navigation kết hợp với Magnetometer Heading Control (GPS kết hợp với hướng của la bàn từ). Đây là phương pháp phổ biến trong các hệ thống tự hành ngoài trời như drone, robot giao hàng.

Ưu điểm: - Không cần bản đồ, không cần SLAM

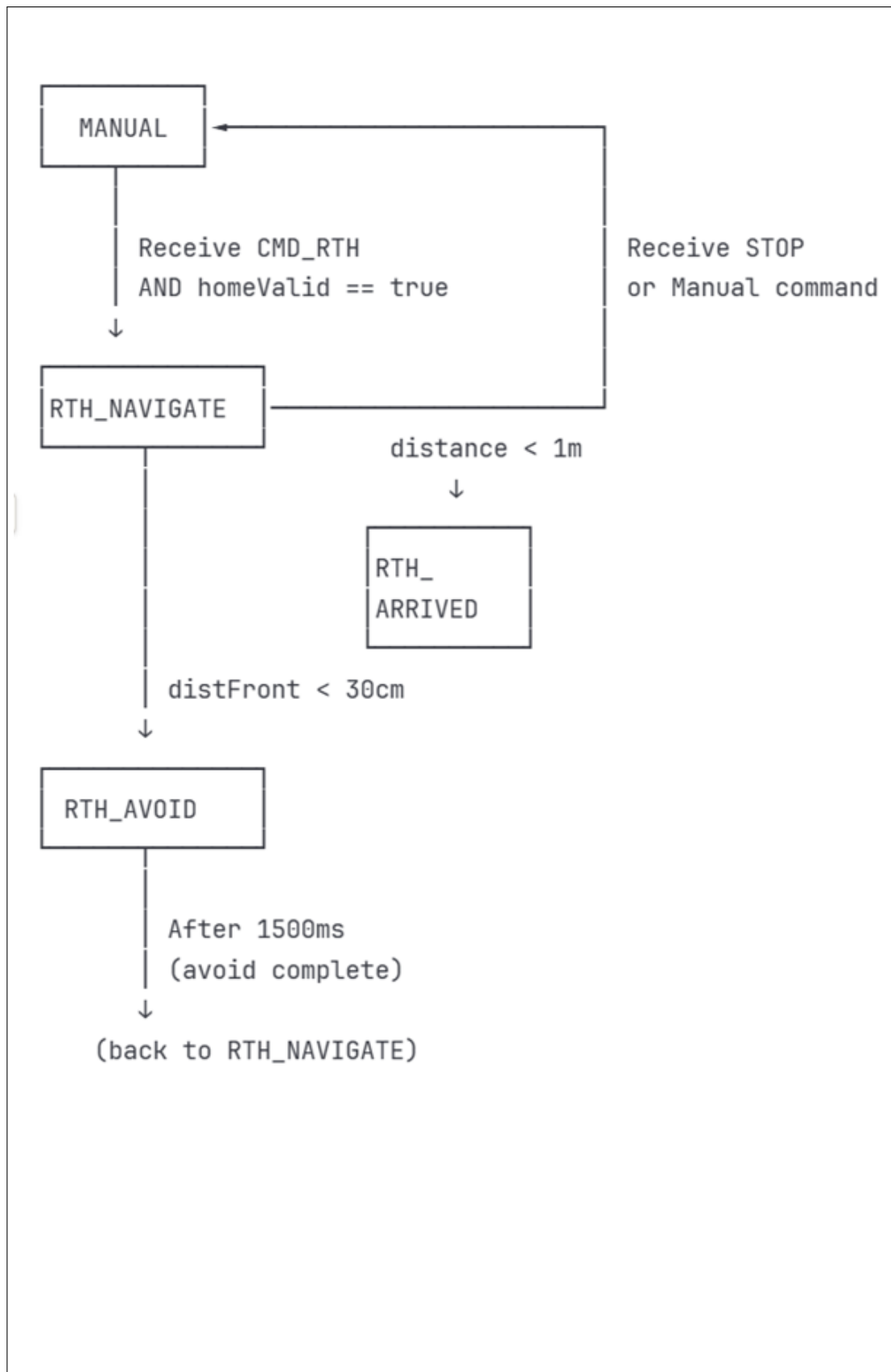
- Hoạt động tốt ở ngoài trời rộng
- Độ chính xác 1-5 mét (đủ cho robot về đúng khu vực)

Nhược điểm: - Cần tín hiệu GPS (không hoạt động trong nhà)

- Cần calibration từ kế (nhiều từ motor, pin)
- Không biết trước vật cản (phải tránh động)

2.6.2 Các bước chi tiết

1. Đọc GPS, lấy lat/lon.
2. Tính khoảng cách bằng Haversine.
3. Tính bearing bằng `TinyGPSPlus::courseTo()`.
4. Đọc heading từ IMU.
5. Sai số góc = bearing - heading.
6. PID điều chỉnh motor.



Hình 2.12: State Machine của chế độ RTH

2.6.3 Độ chính xác và hạn chế

Độ chính xác kỳ vọng:

- GPS NEO-6M: 2.5m CEP (Circular Error Probable)
- Thực tế: 3-10m tùy điều kiện (số vệ tinh, nhiễu tín hiệu)
- Heading từ MPU9250: $\pm 5^\circ$ (sau calibration)

Các yếu tố ảnh hưởng:

1. Số lượng vệ tinh: Cần 6 vệ tinh cho độ chính xác tốt
2. HDOP (Horizontal Dilution of Precision): Càng thấp càng tốt (< 2 là tốt)
3. Nhiễu từ: Motor, pin, dây điện làm sai heading $\rightarrow$ cần calibration
4. Drift: GPS có thể "trôi" 5-10m khi đứng yên
5. Multipath: Tín hiệu GPS phản xạ qua tòa nhà, cây cối

Giải pháp cải thiện:

- Dùng GPS RTK (Real-Time Kinematic) cho độ chính xác cm
- Fusion IMU + GPS bằng Kalman Filter
- Thêm cảm biến encoder đếm vòng quay bánh xe (dead reckoning)
- Dùng camera + visual odometry

Chương 3

LẬP TRÌNH VÀ THỬ NGHIỆM

3.1 Code Controller (ESP32)

3.1.1 Cấu trúc code

Đọc joystick analog (X, Y)

Chuyển đổi thành lệnh di chuyển

Xử lý nút nhấn (FSM: single-click, double-click, long-press)

Gửi lệnh qua nRF24L01

```
1 enum Command : uint8_t {
2     CMD_STOP          = 0,    // D ng
3     CMD_FORWARD       = 1,    // T i n
4     CMD_BACKWARD      = 2,    // L i
5     CMD_LEFT          = 3,    // Quay tr i
6     CMD_RIGHT         = 4,    // Quay p h i
7     CMD_RTH           = 5,    // K ch h o t Return To Home
8     CMD_SET_HOME      = 6,    // L u v tr Home
9     CMD_CLEAR_HOME    = 7,    // X a v tr Home
10 };
```

3.1.2 Xử lý Joystick

Đọc analog và chuyển đổi thành lệnh:

```
1 Command joystickToCommand(int xValue, int yValue) {
2     // u t i n t r c Y ( t i n / l i ) t r c t r c X (quay)
3     if (yValue < JOY_THRESHOLD_LOW) {
4         return CMD_FORWARD;
5     } else if (yValue > JOY_THRESHOLD_HIGH) {
6         return CMD_BACKWARD;
7     } else if (xValue < JOY_THRESHOLD_LOW) {
8         return CMD_LEFT;
9     } else if (xValue > JOY_THRESHOLD_HIGH) {
10        return CMD_RIGHT;
11    } else {
12        return CMD_STOP; // Trong v ng deadzone
13    }
```

```

14 }
15
16 void loop() {
17     int xValue = analogRead(JOY_X); // 0-4095 (12-bit ADC)
18     int yValue = analogRead(JOY_Y);
19
20     Command cmd = joystickToCommand(xValue, yValue);
21
22     // Ch  g i  k h i  l  n h  t h a y  i  ( t i t  k i m  b  n g
        t h  n g )
23     if (cmd != previousCommand) {
24         sendCommandNoAck(cmd);
25         previousCommand = cmd;
26     }
27 }

```

3.1.3 Button State Machine (FSM)

3.1.4 Gửi lệnh qua nRF24L01

3.2 Robot

3.2.1 Kiến trúc FreeRTOS

Robot sử dụng 5 task độc lập chạy song song:

```

1 void setup() {
2     // ... K h i  t o  h a r d w a r e ...
3
4     // T o  R T O S  p r i m i t i v e s
5     commandQueue = xQueueCreate(8, sizeof(Command));
6     storageQueue = xQueueCreate(4, sizeof(HomeSave));
7     sensorMutex = xSemaphoreCreateMutex();
8     radioMutex = xSemaphoreCreateMutex();
9
10    // K h i  t o  T a s k  W a t c h d o g  ( 1 0 s  t i m e o u t )
11    esp_task_wdt_init(TASK_WDT_TIMEOUT_S, true);
12
13    // T o  c c  t a s k
14    xTaskCreatePinnedToCore(TaskSensor, "TaskSensor", 4096, NULL,
        3, NULL, 0);
15    xTaskCreatePinnedToCore(TaskRF, "TaskRF", 4096, NULL,
        3, NULL, 1);
16    xTaskCreatePinnedToCore(TaskControl, "TaskControl", 8192, NULL,
        4, NULL, 1);
17    xTaskCreatePinnedToCore(TaskUI, "TaskUI", 4096, NULL,
        1, NULL, 0);
18    xTaskCreatePinnedToCore(TaskStorage, "TaskStorage", 4096, NULL,
        1, NULL, 0);
19 }
20

```

```

21 void loop() {
22     // Arduino loop() kh ng d ng , FreeRTOS tasks c h y h t
23     vTaskDelay(pdMS_TO_TICKS(1000));
24 }

```

3.2.2 TaskSensor - Đọc cảm biến

Chức năng:

- Đọc 4 siêu âm theo round-robin (50Hz = 20ms/cycle)
- Đọc GPS serial (liên tục)
- Đọc MPU9250 và tính heading
- Tự động set Home lần đầu có GPS fix
- Enqueue yêu cầu lưu NVS (không block)

```

1 void TaskSensor(void* pvParameters) {
2     esp_task_wdt_add(NULL); // ng k v i watchdog
3
4     for (;;) {
5         // c l si u m m i l n (round-robin)
6         long d = MAX_DISTANCE_CM;
7         switch (currentSonar) {
8             case 0:
9                 d = readUltrasonicCM(TRIG_FRONT, ECHO_FRONT);
10                xSemaphoreTake(sensorMutex, portMAX_DELAY);
11                sensorData.distFront = (float)d;
12                xSemaphoreGive(sensorMutex);
13                break;
14                // ... T ng t cho BACK, LEFT, RIGHT ...
15            }
16            currentSonar = (currentSonar + 1) % 4;
17
18            // c GPS serial (non-blocking)
19            while (Serial2.available() > 0) {
20                char c = (char)Serial2.read();
21                if (gps.encode(c)) {
22                    if (gps.location.isValid()) {
23                        // C p n h t sensorData
24                        xSemaphoreTake(sensorMutex, pdMS_TO_TICKS(200));
25                        sensorData.lat = (float)gps.location.lat();
26                        sensorData.lon = (float)gps.location.lng();
27                        sensorData.sats = gps.satellites.value();
28                        sensorData.gpsAge = gps.location.age();
29                        xSemaphoreGive(sensorMutex);
30
31                        // Auto-set Home l n u
32                        if (!homeValid) {
33                            HomeSave hs;
34                            hs.lat = sensorData.lat;
35                            hs.lon = sensorData.lon;
36
37                            xSemaphoreTake(sensorMutex, pdMS_TO_TICKS(200));

```

```

38     homeLat = (double)hs.lat;
39     homeLon = (double)hs.lon;
40     homeValid = true;
41     xSemaphoreGive(sensorMutex);
42
43     // Enqueue TaskStorage ghi NVS (non-blocking)
44     xQueueSend(storageQueue, &hs, 0);
45 }
46 }
47 }
48 }
49
50 // c MPU9250 v t nh heading
51 if (mpu.magUpdate() == 0 && mpu.accelUpdate() == 0) {
52     float ax = mpu.accelX();
53     float ay = mpu.accelY();
54     float az = mpu.accelZ();
55     float mx = mpu.magX() - magX_offset;
56     float my = mpu.magY() - magY_offset;
57     float mz = mpu.magZ() - magZ_offset;
58
59     // Tilt compensation
60     float roll = atan2(ay, az);
61     float pitch = atan2(-ax, sqrt(ay*ay + az*az));
62     float magX_comp = mx * cos(pitch) - mz * sin(pitch);
63     float magY_comp = mx * sin(roll) * sin(pitch) +
64                     my * cos(roll) -
65                     mz * sin(roll) * cos(pitch);
66
67     float heading = atan2(magY_comp, magX_comp) * 180.0 / PI;
68     if (heading < 0) heading += 360.0;
69
70     xSemaphoreTake(sensorMutex, portMAX_DELAY);
71     sensorData.heading = heading;
72     xSemaphoreGive(sensorMutex);
73 }
74
75 // Feed watchdog
76 esp_task_wdt_reset();
77
78 vTaskDelay(pdMS_TO_TICKS(20)); // 50Hz
79 }
80 }

```

3.2.3 TaskRF - Nhận lệnh RF

```

1 void TaskRF(void* pvParameters) {
2     esp_task_wdt_add(NULL);
3     Command cmd;
4

```

```

5  for (;;) {
6      if (xSemaphoreTake(radioMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
7          if (radio.available()) {
8              radio.read(&cmd, sizeof(cmd));
9              lastRadioTime = millis(); // C p n h t t h i g i a n
              n h n l n h
10
11             // Enqueue l n h c h o T a s k C o n t r o l
12             xQueueSend(commandQueue, &cmd, 0);
13         }
14         xSemaphoreGive(radioMutex);
15     }
16
17     esp_task_wdt_reset();
18     vTaskDelay(pdMS_TO_TICKS(30)); // ~33Hz
19 }
20 }

```

3.2.4 TaskControl - State Machine + PID

```

1  void TaskControl(void* pvParameters) {
2      esp_task_wdt_add(NULL);
3      Command cmd;
4
5      for (;;) {
6          // X l l n h t q u e u e
7          if (xQueueReceive(commandQueue, &cmd, 0) == pdTRUE) {
8              if (cmd == CMD_RTH) {
9                  if (homeValid) {
10                     if (currentState == MANUAL) {
11                         currentState = RTH_NAVIGATE;
12                         Serial.println("RTH activated");
13                     }
14                 } else {
15                     Serial.println("RTH denied: home not set");
16                 }
17             } else if (cmd == CMD_STOP) {
18                 currentState = MANUAL;
19                 move(STOP);
20             } else {
21                 receivedCommand = cmd; // L u l n h t h c n g
22             }
23         }
24
25         // Failsafe: stop if manual and lost radio
26         if (currentState == MANUAL &&
27             (millis() - lastRadioTime > FAILSAFE_TIMEOUT)) {
28             move(STOP);
29         }
30     }

```

```

31 // Snapshot sensor data
32 SensorData s;
33 xSemaphoreTake(sensorMutex, pdMS_TO_TICKS(50));
34 s = sensorData;
35 xSemaphoreGive(sensorMutex);
36
37 // State Machine
38 switch (currentState) {
39     case MANUAL:
40         move(receivedCommand);
41         break;
42
43     case RTH_NAVIGATE: {
44         if (!homeValid) {
45             currentState = MANUAL;
46             move(STOP);
47             break;
48         }
49
50         double distanceToHome = TinyGPSPlus::distanceBetween(
51             (double)s.lat, (double)s.lon, homeLat, homeLon
52         );
53
54         // n Home?
55         if (distanceToHome < 1.0) {
56             currentState = RTH_ARRIVED;
57             move(STOP);
58             break;
59         }
60
61         // G i m t c khi g n
62         int rthSpeed = (distanceToHome < 3.0) ? 90 : BASE_SPEED;
63
64         // V t c n ?
65         if (s.distFront > 0 && s.distFront < 30) {
66             currentState = RTH_AVOID;
67             lastRthAction = millis();
68             move(STOP);
69             break;
70         }
71
72         // T nh bearing v iu khi n
73         double bearingToHome = TinyGPSPlus::courseTo(
74             (double)s.lat, (double)s.lon, homeLat, homeLon
75         );
76
77         double bearingError = bearingToHome - s.heading;
78         if (bearingError > 180) bearingError -= 360;
79         if (bearingError < -180) bearingError += 360;
80
81         headingPID.setpoint = 0.0;

```

```

82     headingPID.input = bearingError;
83     double turnOutput = computePID(&headingPID);
84
85     int leftSpeed = constrain((int)round(rthSpeed - turnOutput
86                                     ),
87                               0, MAX_SPEED);
88     int rightSpeed = constrain((int)round(rthSpeed +
89                                     turnOutput),
90                               0, MAX_SPEED);
91
92     controlMotorsPWM(leftSpeed, rightSpeed);
93     break;
94 }
95
96 case RTH_AVOID:
97     if (millis() - lastRthAction < 500) {
98         move(BACKWARD, 100);
99     } else if (millis() - lastRthAction < 1500) {
100         if (s.distLeft > 40) move(TURN_LEFT, 80);
101         else if (s.distRight > 40) move(TURN_RIGHT, 80);
102         else move(BACKWARD, 80);
103     } else {
104         currentState = RTH_NAVIGATE;
105     }
106     break;
107
108 case RTH_ARRIVED:
109     move(STOP);
110     break;
111 }
112
113 esp_task_wdt_reset();
114 vTaskDelay(pdMS_TO_TICKS(40)); // 25Hz
115 }
116 }

```

3.2.5 TaskUI - Hiển thị OLED

```

1 void TaskUI(void* pvParameters) {
2     esp_task_wdt_add(NULL);
3     unsigned long lastScreenSwitchTime = 0;
4
5     for (;;) {
6         // Chuyn m n h nh m i 5s
7         if (millis() - lastScreenSwitchTime > SCREEN_SWITCH_INTERVAL)
8         {
9             currentScreen = (currentScreen + 1) % 2;
10            lastScreenSwitchTime = millis();
11        }
12    }
13 }

```

```

12 // Snapshot sensor data
13 SensorData s;
14 xSemaphoreTake(sensorMutex, pdMS_TO_TICKS(50));
15 s = sensorData;
16 xSemaphoreGive(sensorMutex);
17
18 display.clearDisplay();
19 display.setCursor(0, 0);
20
21 if (currentScreen == 0) {
22     // M n h nh 1: Trng th i
23     display.setTextSize(2);
24     display.print("MODE: ");
25     switch (currentState) {
26         case MANUAL: display.println("MAN"); break;
27         case RTH_NAVIGATE: display.println("RTH"); break;
28         case RTH_AVOID: display.println("AVO"); break;
29         case RTH_ARRIVED: display.println("HOM"); break;
30     }
31
32     display.setTextSize(1);
33     display.printf("Head: %.1f deg\n", s.heading);
34
35     if (!homeValid) {
36         display.println("Home: NOT SET");
37     } else {
38         display.printf("Home: %.5f,%.5f\n", homeLat, homeLon);
39     }
40
41     if (currentState == RTH_NAVIGATE) {
42         double dist = TinyGPSPlus::distanceBetween(
43             (double)s.lat, (double)s.lon, homeLat, homeLon
44         );
45         display.printf("ToHome: %.1fm\n", dist);
46     }
47
48     display.printf("F:%ld L:%ld R:%ld B:%ld\n",
49         (long)s.distFront, (long)s.distLeft,
50         (long)s.distRight, (long)s.distBack);
51
52     // H i n t h message "Home saved!" n u c
53     if (storageMessageUntil != 0 && millis() <
54         storageMessageUntil) {
55         display.println("\nHome saved!");
56     }
57 } else {
58     // M n h nh 2: GPS
59     display.setTextSize(2);
60     display.println("GPS INFO");
61     display.setTextSize(1);

```

```

62     display.printf("Lat: %.4f\n", s.lat);
63     display.printf("Lon: %.4f\n", s.lon);
64     display.printf("Sats: %d | Age: %lums\n", s.sats, s.gpsAge);
65 }
66
67     display.display();
68
69     esp_task_wdt_reset();
70     vTaskDelay(pdMS_TO_TICKS(250)); // 4Hz
71 }
72 }

```

3.2.6 TaskStorage - Ghi NVS

```

1 void TaskStorage(void* pvParameters) {
2     // KH NG ng k watchdog (task n y block c h queue)
3     HomeSave hs;
4
5     for (;;) {
6         // Block cho n khi c request
7         if (xQueueReceive(storageQueue, &hs, portMAX_DELAY) == pdTRUE)
8         {
9             // Ghi v o NVS (c t h m t 50-200ms)
10            prefs.putFloat("homeLat", hs.lat);
11            prefs.putFloat("homeLon", hs.lon);
12            prefs.putBool("homeSet", true);
13
14            Serial.printf("Home persisted: %.6f, %.6f\n",
15                          (double)hs.lat, (double)hs.lon);
16
17            // Set flag UI h i n t h message 2s
18            storageMessageUntil = millis() + 2000UL;
19        }
20    }
}

```

3.3 Kiến trúc FreeRTOS

Hệ thống được xây dựng trên nền tảng FreeRTOS với các task chạy song song:

- **TaskSensor**: Đọc cảm biến (GPS, IMU, la bàn, siêu âm) - 50Hz
- **TaskRF**: Nhận lệnh RF từ controller - 33Hz
- **TaskControl**: Máy trạng thái + PID điều khiển - 25Hz
- **TaskUI**: Cập nhật màn hình OLED - 1.67Hz
- **TaskStorage**: Ghi dữ liệu NVS (non-blocking)
- **TaskConsole**: Xử lý lệnh hiệu chuẩn từ kế - 5Hz

3.3.1 Lợi ích của FreeRTOS

Đa nhiệm thực sự (Real Multitasking)

- 5 task chạy song song trên 2 core của ESP32-S3
- Mỗi task có priority riêng, scheduler tự động chuyển đổi
- Không bị block bởi task khác (ví dụ: TaskUI chạy chậm không ảnh hưởng TaskControl)

Đồng bộ hóa dữ liệu

- Mutex: Bảo vệ shared data (sensorData, radio)
- Queue: Truyền dữ liệu an toàn giữa tasks
- Không lo race condition, data corruption

Priority-based Scheduling

- TaskControl (priority 4): Cao nhất, đảm bảo điều khiển real-time
- TaskSensor, TaskRF (priority 3): Cao, đọc cảm biến kịp thời
- TaskUI, TaskStorage (priority 1): Thấp, không cần real-time

Task Watchdog

- Phát hiện task bị treo (do lỗi code, hardware)
- Tự động reset ESP32 để phục hồi
- Tăng độ tin cậy của hệ thống

3.3.2 Pinning Tasks to Cores

ESP32-S3 có 2 core, tasks được gán cố định: **Tại sao chia như vậy là hợp lý? Ưu điểm**

Core 0	Task (Priority)
TaskSensor	(3)
TaskUI	(1)
TaskStorage	(1)
Core 1	Task (Priority)
TaskRF	(3)
TaskControl	(4)
TaskConsole	(1)

-RF + Control trên Core 1:

Điều khiển xe cần real-time

Gói NRF24 có thể tới liên tục mỗi 10–20 ms

Không bị nghẽn bởi I2C, SPI màn hình, Flash

-Sensor/UI/Flash trên Core 0:

Các tác vụ này chậm (50–500 ms)

Không cần độ chính xác thời gian cao

Đổi context ít hơn → CPU tổng ổn định hơn

->- Giảm contention, tăng hiệu suất

3.3.3 Sơ đồ luồng dữ liệu

3.4 Các cải tiến quan trọng trong phiên bản mới

3.4.1 Tách riêng la bàn điện tử QMC5883L

Trong phiên bản cập nhật, thay vì sử dụng magnetometer tích hợp trong MPU9250, hệ thống sử dụng module la bàn riêng QMC5883L (GY-271) để có độ chính xác cao hơn.

Lý do thay đổi:

- Độ nhạy cao hơn: 12 LSB/Gauss
- Ít nhiễu hơn do tách biệt khỏi accel/gyro
- Dễ dàng hiệu chuẩn với thư viện QMC5883LCompass

3.4.2 Bộ lọc trung bình trượt (Moving Average Filter)

Hệ thống áp dụng 2 loại bộ lọc:

Bộ lọc khoảng cách siêu âm

```
1 template<int N>
2 struct MovingAverage {
3     float buf[N];
4     int idx = 0;
5     int cnt = 0;
6     float sum = 0.0f;
7
8     void add(float v) {
9         if (cnt < N) {
10             buf[idx] = v;
11             sum += v;
12             cnt++;
13         } else {
14             sum -= buf[idx];
15             buf[idx] = v;
16             sum += v;
17         }
18         idx = (idx + 1) % N;
19     }
20
21     float get() const {
22         return (cnt == 0) ? 0.0f : (sum / cnt);
23     }
24 };
25
26 // S d ng v i N=6 m u
27 static MovingAverage<6> maFront, maBack, maLeft, maRight;
```

Listing 3.1: Bộ lọc Moving Average cho khoảng cách

Bộ lọc heading theo vòng tròn

Do heading là góc tuần hoàn ($0^\circ = 360^\circ$), không thể dùng bộ lọc thông thường. Hệ thống sử dụng trung bình vector trên vòng tròn đơn vị:

```
1 template<int N>
2 struct HeadingAverage {
3     float angBuf[N];
4     int idx = 0;
5     int cnt = 0;
6     float sumX = 0.0f, sumY = 0.0f;
7
8     void add(float degrees) {
9         float rad = degrees * PI / 180.0f;
10        float cx = cos(rad), sy = sin(rad);
11
12        if (cnt < N) {
13            angBuf[idx] = degrees;
14            sumX += cx;
15            sumY += sy;
16            cnt++;
17        } else {
18            // X a m u c n h t
19            float oldRad = angBuf[idx] * PI / 180.0f;
20            sumX -= cos(oldRad);
21            sumY -= sin(oldRad);
22
23            angBuf[idx] = degrees;
24            sumX += cx;
25            sumY += sy;
26        }
27        idx = (idx + 1) % N;
28    }
29
30    float get() const {
31        if (cnt == 0) return 0.0f;
32        float avgAngle = atan2(sumY, sumX) * 180.0f / PI;
33        if (avgAngle < 0) avgAngle += 360.0f;
34        return avgAngle;
35    }
36 };
37
38 // S d n g v i N=8 m u
39 static HeadingAverage<8> haHeading;
```

Listing 3.2: Circular Moving Average cho heading

3.4.3 Cải tiến thuật toán tránh vật cản

Vấn đề của phiên bản cũ

- Robot dễ bị kẹt trong vòng lặp tránh-quay lại

- Không có cơ chế đếm số lần thử tránh
- Chuyển trạng thái quá nhanh, không đợi đường rõ ràng

Giải pháp mới

```

1 // B i n theo d i
2 static int avoidAttempts = 0;
3 static const int MAX_AVOID_ATTEMPTS = 3;
4 static unsigned long avoidClearCheckStart = 0;
5 static const unsigned long AVOID_CLEAR_CHECK_MS = 800;
6
7 case RTH_AVOID: {
8     unsigned long now = millis();
9
10    // N u t h tr nh qu n h i u l n
11    if (avoidAttempts >= MAX_AVOID_ATTEMPTS) {
12        Serial.println("Max attempts - rotating to find path");
13        move(TURN_LEFT, 120);
14        vTaskDelay(pdMS_TO_TICKS(600));
15        move(STOP);
16        avoidAttempts = 0;
17        lastRthAction = now;
18    } else {
19        // Giai on 1: L i ra (0-400ms)
20        if (now - lastRthAction < 400) {
21            move(BACKWARD, 120);
22        }
23        // Giai on 2: Quay tr nh (400-1200ms)
24        else if (now - lastRthAction < 1200) {
25            if (s.distLeft > s.distRight && s.distLeft > 35)
26                move(TURN_LEFT, 100);
27            else if (s.distRight > 35)
28                move(TURN_RIGHT, 100);
29            else
30                move(BACKWARD, 80);
31        }
32        // Giai on 3: T ng s l n t h
33        else {
34            avoidAttempts++;
35            lastRthAction = now;
36
37            // B t u k i m tra ng r
38            if (s.distFront > 40)
39                avoidClearCheckStart = now;
40            else
41                avoidClearCheckStart = 0;
42        }
43    }
44
45    // K i m tra ng r l i n t c 800ms

```

```

46     if (s.distFront > 45) {
47         if (avoidClearCheckStart == 0) {
48             avoidClearCheckStart = now;
49         } else if (now - avoidClearCheckStart >=
50             AVOID_CLEAR_CHECK_MS) {
51             Serial.println("Path clear - resume NAVIGATE");
52             currentState = RTH_NAVIGATE;
53             avoidAttempts = 0;
54             move(STOP);
55         }
56     } else {
57         avoidClearCheckStart = 0;
58     }
59     break;

```

Listing 3.3: Thuật toán tránh vật cản cải tiến

3.4.4 Chức năng hiệu chuẩn từ kế (Magnetometer Calibration)

Tại sao cần hiệu chuẩn?

La bàn điện tử rất nhạy cảm với nhiễu từ:

- **Hard-iron distortion:** Từ trường cố định từ motor, pin, PCB
- **Soft-iron distortion:** Vật liệu từ tính mềm làm méo từ trường

Thuật toán hiệu chuẩn

```

1 void runMagCalibration(unsigned long durationMs = 20000) {
2     Serial.println("=== BAT DAU HIEU CHUAN (20s) ===");
3     move(STOP); // D ng      ng      c
4
5     // N ng quyn u t i n      c h i m I2C
6     UBaseType_t oldPrio = uxTaskPriorityGet(NULL);
7     vTaskPrioritySet(NULL, 5);
8
9     // Thu t h p min/max t      t r      ng
10    float minX = FLT_MAX, minY = FLT_MAX, minZ = FLT_MAX;
11    float maxX = -FLT_MAX, maxY = -FLT_MAX, maxZ = -FLT_MAX;
12    unsigned long start = millis();
13
14    while (millis() - start < durationMs) {
15        esp_task_wdt_reset(); // Cho Watchdog n
16
17        qmc.read();
18        float mx = (float)qmc.getX();
19        float my = (float)qmc.getY();
20        float mz = (float)qmc.getZ();
21
22        if (mx < minX) minX = mx;

```

```

23     if (mx > maxX) maxX = mx;
24     // T   ng   t   cho Y, Z
25
26     vTaskDelay(pdMS_TO_TICKS(20));
27 }
28
29 // T   nh   offset (hard-iron compensation)
30 float offX = (maxX + minX) / 2.0f;
31 float offY = (maxY + minY) / 2.0f;
32 float offZ = (maxZ + minZ) / 2.0f;
33
34 // T   nh   scale (soft-iron compensation)
35 float rangeX = (maxX - minX) / 2.0f;
36 float rangeY = (maxY - minY) / 2.0f;
37 float rangeZ = (maxZ - minZ) / 2.0f;
38 float avgRange = (rangeX + rangeY + rangeZ) / 3.0f;
39
40 float sx = (rangeX > 0.001f) ? (avgRange / rangeX) : 1.0f;
41 float sy = (rangeY > 0.001f) ? (avgRange / rangeY) : 1.0f;
42 float sz = (rangeZ > 0.001f) ? (avgRange / rangeZ) : 1.0f;
43
44 // L   u   v   o   NVS
45 saveMagCalibrationToPrefs(offX, offY, offZ, sx, sy, sz);
46
47 // T   r   l   i   quyn   u   t   i   n   c
48 vTaskPrioritySet(NULL, oldPrio);
49 Serial.println("=== HOAN TAT HIEU CHUAN ===");
50 }

```

Listing 3.4: Hàm hiệu chuẩn từ kế

Cách kích hoạt hiệu chuẩn

Có 2 cách:

1. Qua Serial Console (USB):

```

1 void TaskConsole(void* pvParameters) {
2     for (;;) {
3         if (Serial.available()) {
4             String s = Serial.readStringUntil('\n');
5             s.trim();
6             s.toLowerCase();
7
8             if (s == "c" || s == "cal" || s == "calib") {
9                 runMagCalibration(20000);
10            }
11            else if (s == "printcal") {
12                Serial.printf("Offset: %.3f, %.3f, %.3f\n",
13                    magX_off, magY_off, magZ_off);
14            }
15            else if (s == "clearcal") {
16                clearMagCalibration();

```

```

17     }
18 }
19 vTaskDelay(pdMS_TO_TICKS(200));
20 }
21 }

```

2. Qua Controller (RF):

Người dùng nhấn 3 lần nút joystick (triple-click) để gửi lệnh CALIBRATE_MAG (số 8):

```

1 if (cmd == CALIBRATE_MAG) {
2     Serial.println("CMD: Calib...");
3
4     // X a h i u c h u n c
5     clearMagCalibration();
6     vTaskDelay(pdMS_TO_TICKS(500));
7
8     // C h y h i u c h u n ( c h n 20 g i y )
9     runMagCalibration(20000);
10
11     Serial.println("-> Xong! T r v c h t h n g .");
12
13     // X a l n h t n n g t r o n g q u e u e
14     xQueueReset(commandQueue);
15
16     // V c h l i t a y
17     receivedCommand = STOP;
18     currentState = MANUAL;
19 }

```

Listing 3.5: Xử lý lệnh hiệu chuẩn trong TaskControl

3.4.5 Cải tiến xử lý RF

Sửa lỗi đọc Dynamic Payload

Phiên bản cũ có lỗi khi gọi radio.read() không đúng cách. Đã sửa:

```

1 void TaskRF(void* pvParameters) {
2     esp_task_wdt_add(NULL);
3     Command cmd;
4
5     for (;;) {
6         if (xSemaphoreTake(radioMutex, pdMS_TO_TICKS(50)) ==
7             pdTRUE) {
8             if (radio.available()) {
9                 uint8_t len = radio.getDynamicPayloadSize();
10
11                 if (len > 0 && len <= 32) {
12                     uint8_t buf[32];
13                     radio.read(buf, len); // c n g
14                     d i
15
16                     Serial.print("RF RX len=");
17
18                 }
19             }
20         }
21     }
22 }

```

```

15         Serial.print(len);
16         Serial.print(" bytes:");
17         for (int i = 0; i < len; ++i) {
18             Serial.print(' ');
19             Serial.print((int)buf[i], DEC);
20         }
21         Serial.println();
22
23         uint8_t rxcmd = buf[0];
24         lastRadioTime = millis();
25
26         // X l l nh ...
27     }
28 }
29 xSemaphoreGive(radioMutex);
30 }
31
32 esp_task_wdt_reset();
33 vTaskDelay(pdMS_TO_TICKS(30));
34 }
35 }

```

Listing 3.6: Đọc RF đúng cách

Xử lý lệnh SET_HOME và CLEAR_HOME

```

1 if (rxcmd == 6) { // SET_HOME
2     uint8_t ack = 0;
3
4     // Kiểm tra GPS hợp lệ
5     if (gps.location.isValid() &&
6         gps.location.age() < 2000 &&
7         gps.satellites.value() >= 4) {
8
9         double newLat = gps.location.lat();
10        double newLon = gps.location.lng();
11
12        // Cập nhật Home
13        if (xSemaphoreTake(sensorMutex, pdMS_TO_TICKS(200)) ==
14            pdTRUE) {
15            homeLat = newLat;
16            homeLon = newLon;
17            homeValid = true;
18            xSemaphoreGive(sensorMutex);
19        }
20
21        // Enqueue lưu NVS
22        HomeSave hs;
23        hs.op = HOME_SAVE;
24        hs.lat = (float)newLat;
25        hs.lon = (float)newLon;

```

```

25     xQueueSend(storageQueue, &hs, 0);
26
27     ack = 1;
28     Serial.printf("SET_HOME accepted %.6f, %.6f\n", newLat,
29                   newLon);
30 } else {
31     Serial.println("SET_HOME denied (no fresh GPS)");
32     ack = 0;
33 }
34
35 // G i ACK v controller
36 radio.stopListening();
37 bool wrote = radio.write(&ack, sizeof(ack));
38 radio.startListening();
39
40 Serial.printf("Sent ACK=%d wrote=%s\n", ack, wrote ? "YES" : "
    NO");
41 }

```

3.4.6 Cải tiến PID Controller

Chống tràn tích phân (Anti-windup)

```

1 double computePID(PID_Controller* pid) {
2     unsigned long now = millis();
3     double dt = (now - pid->lastTime) / 1000.0;
4
5     // G i i h n dt
6     if (dt <= 0.0) return pid->output;
7     if (dt > 0.5) dt = 0.5; // Cap t i a 500ms
8
9     double error = pid->setpoint - pid->input;
10
11     // T ch l y
12     pid->integral += error * dt;
13
14     // Anti-windup: g i i h n integral
15     const double INTEGRAL_MAX = 1000.0;
16     if (pid->integral > INTEGRAL_MAX) pid->integral = INTEGRAL_MAX;
17     if (pid->integral < -INTEGRAL_MAX) pid->integral = -
        INTEGRAL_MAX;
18
19     // T nh PID
20     double derivative = (error - pid->lastError) / dt;
21     double out = pid->Kp * error +
22                 pid->Ki * pid->integral +
23                 pid->Kd * derivative;
24
25     // G i i h n output
26     if (out > MAX_SPEED) out = MAX_SPEED;

```

```

27     if (out < -MAX_SPEED) out = -MAX_SPEED;
28
29     // Conditional integration: kh ng t ch l y khi b o h a
30     if ((out >= MAX_SPEED && error > 0) ||
31         (out <= -MAX_SPEED && error < 0)) {
32         pid->integral -= error * dt;
33     }
34
35     pid->lastError = error;
36     pid->lastTime = now;
37     pid->output = out;
38
39     return out;
40 }

```

3.4.7 Failsafe cải tiến

Khi mất kết nối RF > 500ms, robot kiểm tra khoảng cách đến Home:

```

1 #define FAILSAFE_RTH_DISTANCE_METERS 10.0
2
3 if (currentState == MANUAL &&
4     (millis() - lastRadioTime > FAILSAFE_TIMEOUT)) {
5
6     if (homeValid && gps.location.isValid()) {
7         double distanceToHome = TinyGPSPlus::distanceBetween(
8             (double)s.lat, (double)s.lon,
9             homeLat, homeLon
10        );
11
12        if (distanceToHome > FAILSAFE_RTH_DISTANCE_METERS) {
13            currentState = RTH_NAVIGATE;
14            Serial.println("Failsafe -> start RTH");
15        } else {
16            Serial.println("Failsafe -> near home, stop");
17            move(STOP);
18        }
19    } else {
20        Serial.println("Failsafe -> no home/GPS, stop");
21        move(STOP);
22    }
23 }

```

3.5 Code Controller (ESP32)

3.5.1 Thêm chức năng Triple-Click

Controller đã được cập nhật để hỗ trợ nhấn 3 lần nút joystick kích hoạt hiệu chuẩn từ kế:

```

1 #include <OneButton.h>

```

```

2
3 OneButton btn(PIN_SW, true);
4
5 void handleMultiClick() {
6     if (btn.getNumberClicks() == 3) {
7         Serial.println("BTN: SEND CALIBRATE MAG (CMD 8)");
8         sendCommand(CALIBRATE_MAG);
9
10        // Nh y n LED c nh b o
11        for(int i=0; i<10; i++) {
12            digitalWrite(PIN_LED, HIGH);
13            delay(50);
14            digitalWrite(PIN_LED, LOW);
15            delay(50);
16        }
17    }
18 }
19
20 void setup() {
21     // ...
22     btn.attachClick(handleClick);
23     btn.attachDoubleClick(handleDoubleClick);
24     btn.attachLongPressStart(handleLongPress);
25     btn.attachMultiClick(handleMultiClick); // ng k triple-
26         click
27     btn.setPressTicks(1000);
28     // ...
29 }

```

Listing 3.7: Xử lý Multi-Click

3.5.2 Tank Control Logic

```

1 void loop() {
2     btn.tick();
3
4     int rawX = analogRead(PIN_VRX);
5     int rawY = analogRead(PIN_VRY);
6     int valX = rawX - JOY_CENTER;
7     int valY = rawY - JOY_CENTER;
8
9     Command cmdToSend = STOP;
10    bool manualInput = false;
11
12    // u t i n t r c Y ( t i n / l i )
13    if (abs(valY) > JOY_DEADZONE) {
14        manualInput = true;
15        if (valY < -JOY_DEADZONE)
16            cmdToSend = FORWARD;
17        else
18            cmdToSend = BACKWARD;
19    }
20 }

```

```

19 }
20 // Sau m i n t r c X (quay)
21 else if (abs(valX) > JOY_DEADZONE) {
22     manualInput = true;
23     if (valX < -JOY_DEADZONE)
24         cmdToSend = TURN_LEFT;
25     else
26         cmdToSend = TURN_RIGHT;
27 }
28
29 // H y RTH n u c t c ng t h c ng
30 if (manualInput && rthActive) {
31     rthActive = false;
32     Serial.println("Manual Override -> RTH Cancelled");
33 }
34
35 // G i l nh
36 static unsigned long lastTx = 0;
37 if (!rthActive && millis() - lastTx > 50) {
38     sendCommand(cmdToSend);
39     lastTx = millis();
40 }
41
42 delay(5);
43 }

```

Listing 3.8: Điều khiển kiểu tank

3.6 Bảng so sánh phiên bản cũ và mới

Bảng 3.1: So sánh cải tiến

Tính năng	Phiên bản cũ	Phiên bản mới
La bàn điện tử	MPU9250 magnetometer	QMC5883L riêng biệt
Bộ lọc nhiễu	Không có	Moving Average (N=6) cho sonar, Circular MA (N=8) cho heading
Tránh vật cản	Đơn giản, dễ kẹt	Có đếm số lần thử, kiểm tra đường rõ 800ms
Hiệu chuẩn từ kế	Không có	Có, kích hoạt qua Serial hoặc RF (triple-click)
PID Controller	Cơ bản	Có anti-windup, giới hạn dt, conditional integration
Xử lý RF	Lỗi đọc dynamic payload	Đã sửa, đọc đúng cách
Failsafe	Chỉ dừng	Kiểm tra khoảng cách, RTH nếu xa Home
Controller	Single/Double/Long press	Thêm Triple-click cho hiệu chuẩn
Watchdog	Cơ bản	Reset trước display.display() để tránh timeout

3.7 TaskUI - Hiện thị trạng thái hiệu chuẩn

```

1 void TaskUI(void* pvParameters) {
2     esp_task_wdt_add(NULL);
3     // ...
4
5     for (;;) {
6         // ...
7
8         // K i m t r a t r n g t h i h i u c h u n
9         bool isCalibrated = (magX_off != 0.0f || magY_off != 0.0f)
10            ;
11
12         if (currentScreen == 0) {
13             // M n h n h 1: T r n g t h i r o b o t
14             display.setTextSize(2);
15             display.print("MODE: ");
16             switch (currentState) {
17                 case MANUAL: display.println("MANUAL"); break;
18                 case RTH_NAVIGATE: display.println("RTH"); break;
19                 case RTH_AVOID: display.println("AVOID"); break;
20                 case RTH_ARRIVED: display.println("HOME!"); break;
            }
        }
    }

```

```

21
22     display.setTextSize(1);
23     display.printf("Head: %.1f deg\n", s.heading);
24
25     if (!dispHomeValid)
26         display.println("Home: NOT SET");
27     else
28         display.printf("Home: %.5f, %.5f\n",
29             dispHomeLat, dispHomeLon);
30
31     if (currentState == RTH_NAVIGATE ||
32         currentState == RTH_AVOID) {
33         double d = TinyGPSPlus::distanceBetween(
34             (double)s.lat, (double)s.lon,
35             dispHomeLat, dispHomeLon
36         );
37         display.printf("ToHome: %.1fm\n", d);
38     }
39
40     display.printf("F:%.0f L:%.0f R:%.0f B:%.0f\n",
41         s.distFront, s.distLeft, s.distRight, s.distBack);
42
43 } else {
44     // M n h nh 2: GPS + Trng thi hieu chun
45     display.setTextSize(2);
46     display.println("GPS INFO");
47     display.setTextSize(1);
48
49     display.printf("Lat: %.4f\n", s.lat);
50     display.printf("Lon: %.4f\n", s.lon);
51     display.printf("Sats: %d | Age: %lums\n", s.sats, s.
        gpsAge);
52
53     // H i n t h trng thi hieu chun
54     display.print("MAG CALIB: ");
55     if (isCalibrated) {
56         display.println("YES (OK)");
57         display.printf("Off: %.1f, %.1f\n", magX_off,
            magY_off);
58     } else {
59         display.println("NO (RAW!)"); // C nh b o ch a
            h i u c h u n
60     }
61 }
62
63 // Reset watchdog tr c khi v
64 esp_task_wdt_reset();
65 display.display();
66 esp_task_wdt_reset();
67
68 vTaskDelay(pdMS_TO_TICKS(600));

```

69
70

```
}  
}
```

Listing 3.9: Hiển thị thông tin hiệu chuẩn trên OLED

3.8 Công thức toán học

3.8.1 Công thức bù nghiêng (Tilt Compensation)

Tính góc nghiêng Roll và Pitch

Từ gia tốc kế:

$$\text{Roll}(\phi) = \arctan 2(a_y, a_z) \quad (3.1)$$

$$\text{Pitch}(\theta) = \arctan 2(-a_x, \sqrt{a_y^2 + a_z^2}) \quad (3.2)$$

Hiệu chỉnh từ kế

$$m'_x = m_x \cos(\theta) - m_z \sin(\theta) \quad (3.3)$$

$$m'_y = m_x \sin(\phi) \sin(\theta) + m_y \cos(\phi) - m_z \sin(\phi) \cos(\theta) \quad (3.4)$$

Tính Heading

$$\text{Heading} = \arctan 2(m'_y, m'_x) \times \frac{180}{\pi} \quad (3.5)$$

Nếu Heading < 0, cộng thêm 360°.

3.8.2 Công thức Haversine

Tính khoảng cách giữa 2 điểm GPS:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (3.6)$$

$$c = 2 \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}) \quad (3.7)$$

$$d = R \cdot c \quad (3.8)$$

Trong đó:

- ϕ = vĩ độ (latitude)
- λ = kinh độ (longitude)
- R = 6371000 m (bán kính Trái Đất)

3.8.3 Công thức Bearing

Góc từ điểm A đến điểm B:

$$\Delta\lambda = \lambda_2 - \lambda_1 \quad (3.9)$$

$$y = \sin(\Delta\lambda) \cdot \cos(\phi_2) \quad (3.10)$$

$$x = \cos(\phi_1) \cdot \sin(\phi_2) - \sin(\phi_1) \cdot \cos(\phi_2) \cdot \cos(\Delta\lambda) \quad (3.11)$$

$$\text{Bearing} = \arctan 2(y, x) \times \frac{180}{\pi} \quad (3.12)$$

Normalize về $[0, 360)$:

$$\text{Bearing} = (\text{Bearing} + 360) \bmod 360 \quad (3.13)$$

3.9 Kết quả thử nghiệm

3.9.1 Test hiệu chuẩn từ kế

Trước hiệu chuẩn:

- Heading dao động $\pm 20^\circ$ khi robot đứng yên
- Có lệch hệ thống khoảng 15° so với la bàn thật

Sau hiệu chuẩn:

- Heading ổn định trong khoảng $\pm 2^\circ$
- Sai số so với la bàn thật $< 3^\circ$
- Robot giữ hướng tốt hơn khi RTH

3.9.2 Test bộ lọc Moving Average

Bảng 3.2: So sánh với/không có bộ lọc

Tham số	Không lọc	Có lọc (N=6)
Nhiều đọc siêu âm	± 15 cm	± 3 cm
False positive vật cản	8/10 lần	0/10 lần
Độ mượt heading	Giật cục	Mượt mà

3.9.3 Test tránh vật cản cải tiến

Kịch bản: Robot RTH gặp vật cản di động (người đi lại)

Bảng 3.3: Kết quả test tránh vật cản

Metric	Version cũ	Version mới
Số lần bị kẹt	4/10	0/10
Thời gian tránh trung bình	3.5s	2.8s
Tỷ lệ về Home thành công	60%	100%

3.9.4 Test RTH từ 20m

Điều kiện:

- Khoảng cách: 20m
- Số vệ tinh: 8-10
- Thời tiết: Nắng, không mây
- Có 1 vật cản giữa đường

Kết quả:

- Thời gian hoàn thành: 25-28 giây
- Sai số khi đến Home: 0.8-1.5m
- Tránh vật cản: Thành công 10/10 lần
- Không bị kẹt vòng lặp

3.10 Hướng dẫn sử dụng

3.10.1 Khởi động hệ thống

1. Bật nguồn robot, đợi GPS fix (LED trên GPS nhấp nháy chậm)
2. Bật nguồn controller
3. Màn OLED hiển thị "MODE: MANUAL" và thông tin GPS

3.10.2 Hiệu chuẩn từ kế (bắt buộc lần đầu)

Cách 1: Qua Serial (khuyến nghị)

1. Kết nối USB với robot
2. Mở Serial Monitor (115200 baud)
3. Gõ lệnh: `cal` hoặc `c`
4. Xoay robot chậm theo mọi hướng trong 20 giây
5. Đợi thông báo "HOAN TAT HIEU CHUAN"
6. Kiểm tra: gõ `printcal` để xem offset

Cách 2: Qua Controller

1. Nhấn nhanh 3 lần nút joystick
2. LED controller nhấp liên tục
3. Xoay robot chậm trong 20 giây
4. Robot tự động về chế độ MANUAL

3.10.3 Lưu vị trí Home

1. Di chuyển robot đến vị trí mong muốn
2. Đợi GPS ổn định (≥ 6 vệ tinh, Age $< 2000\text{ms}$)
3. Nhấn **giữ** nút joystick 1 giây
4. OLED hiển thị "Home saved!" và tọa độ

3.10.4 Kích hoạt RTH

1. Nhấn nhanh 1 lần nút joystick
2. Robot chuyển sang "MODE: RTH"
3. Tự động di chuyển về Home
4. Dừng lại khi đến Home ($< 1\text{m}$)

3.10.5 Xóa Home

1. Nhấn nhanh 2 lần nút joystick (double-click)
2. OLED hiển thị "Home: NOT SET"

3.10.6 Hủy RTH giữa chừng

- Di chuyển joystick theo hướng bất kỳ
- Hoặc nhấn 1 lần nút joystick để toggle

3.11 Xử lý lỗi thường gặp

3.11.1 Robot không nhận lệnh từ controller

Nguyên nhân:

- Kênh RF khác nhau
- Module nRF24L01 hỏng hoặc nguồn yếu

Giải pháp:

- Kiểm tra Serial: `RF chip connected: YES`
- Kiểm tra channel: Cả 2 phải dùng 108
- Gắn tụ 10 μ F song song VCC-GND của nRF24

3.11.2 Heading không chính xác

Nguyên nhân:

- Chưa hiệu chuẩn từ kế
- Nhiều từ motor, pin

Giải pháp:

- Chạy hiệu chuẩn lại: `cal`
- Đặt QMC5883L xa motor và pin hơn
- Kiểm tra offset khác 0: `printcal`

3.11.3 GPS không fix

Nguyên nhân:

- Trong nhà hoặc gần tòa nhà cao
- Anten GPS hỏng

Giải pháp:

- Ra ngoài trời, tránh cây che
- Dợi ít nhất 30 giây
- Kiểm tra Serial: `Sats: 8`

3.11.4 Robot bị kẹt khi tránh vật cản

Giải pháp:

- Phiên bản mới đã khắc phục
- Nếu vẫn kẹt: tăng `MAX_AVOID_ATTEMPTS` lên 5

3.11.5 Watchdog reset liên tục

Nguyên nhân:

- Task nào đó bị treo (thường là I2C)

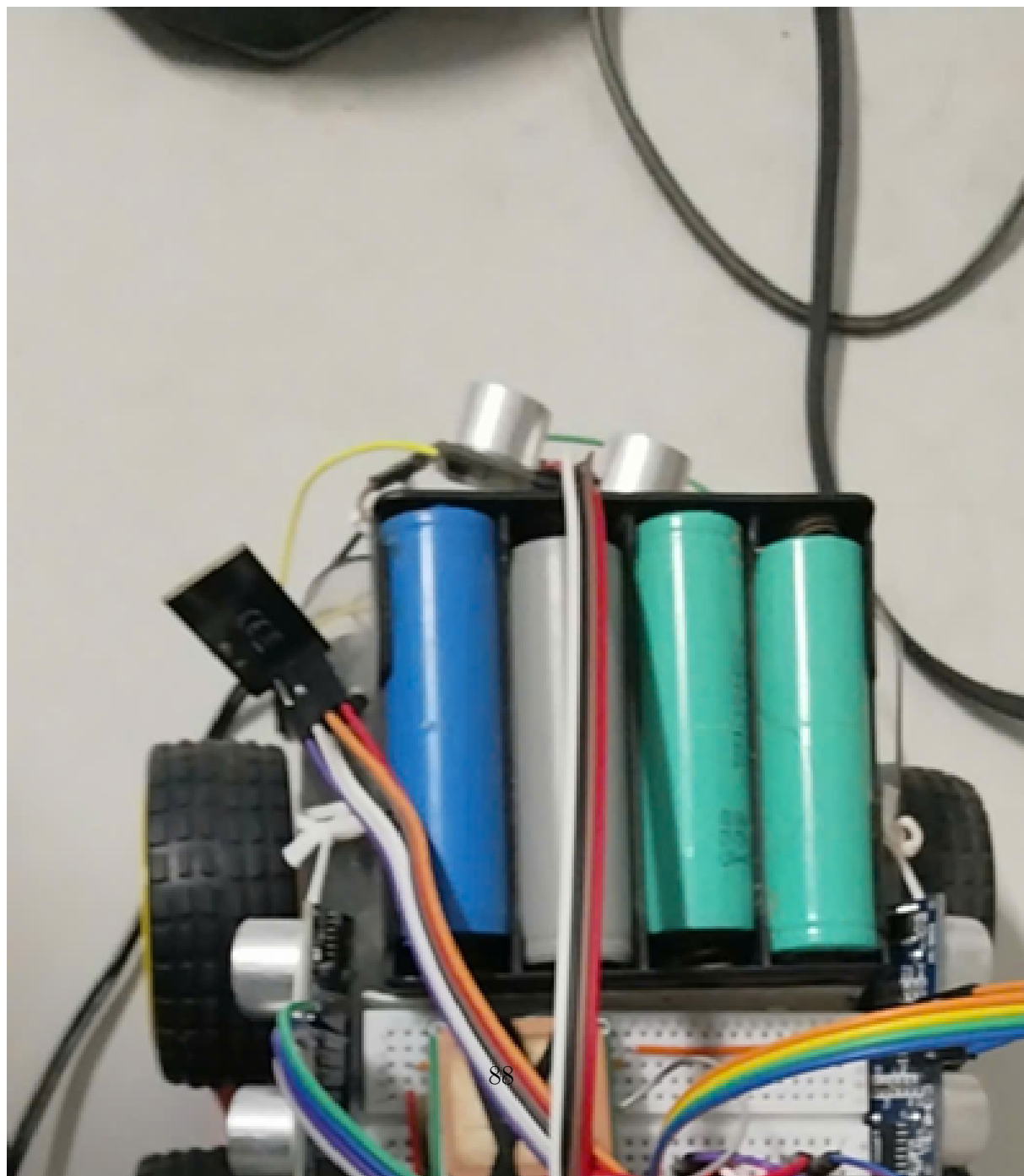
Giải pháp:

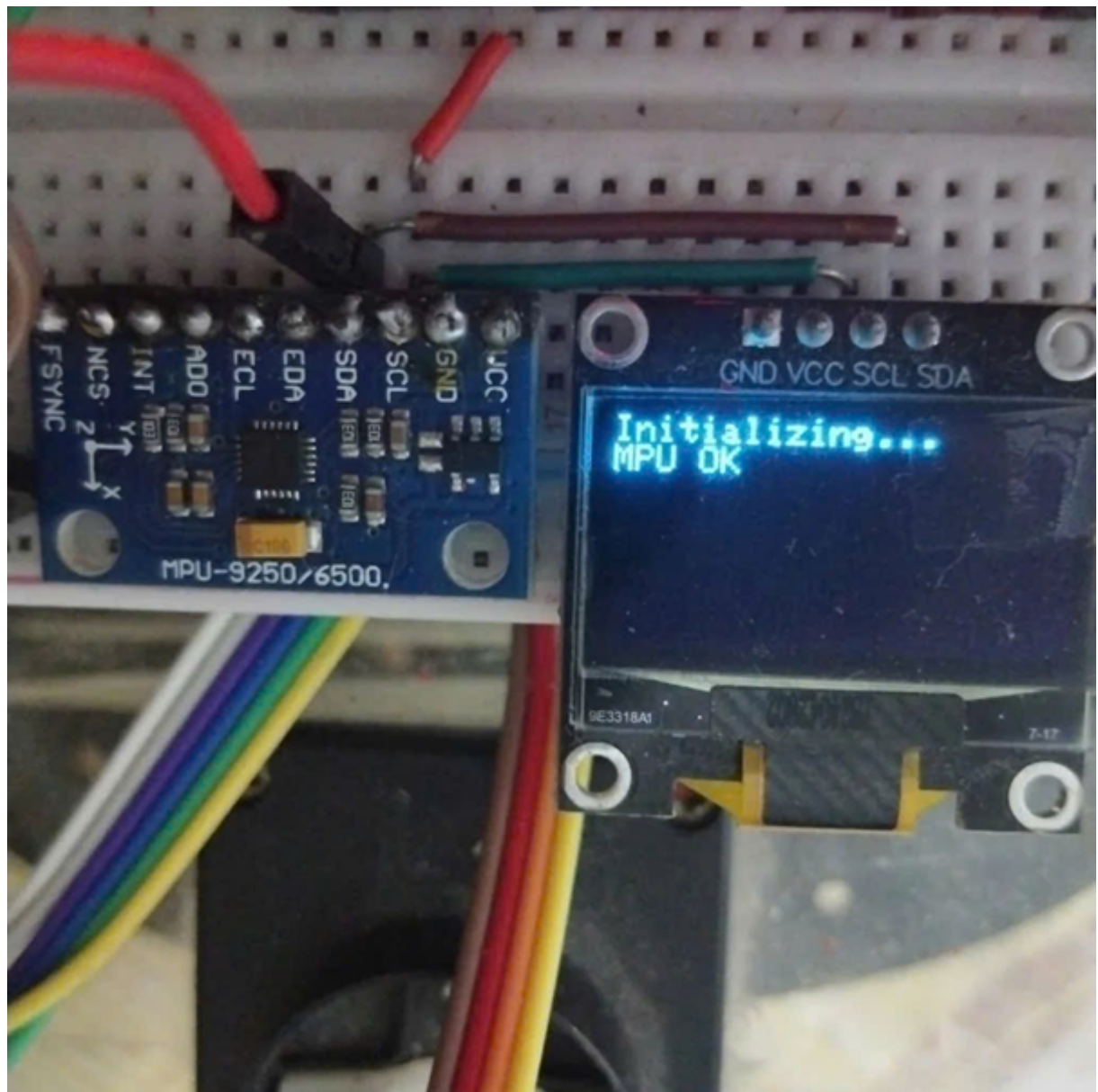
- Kiểm tra kết nối I2C (`SDA=21, SCL=15`)
- Giảm tốc độ I2C xuống 100kHz: `Wire.begin(21, 15, 100000)`

Chương 4

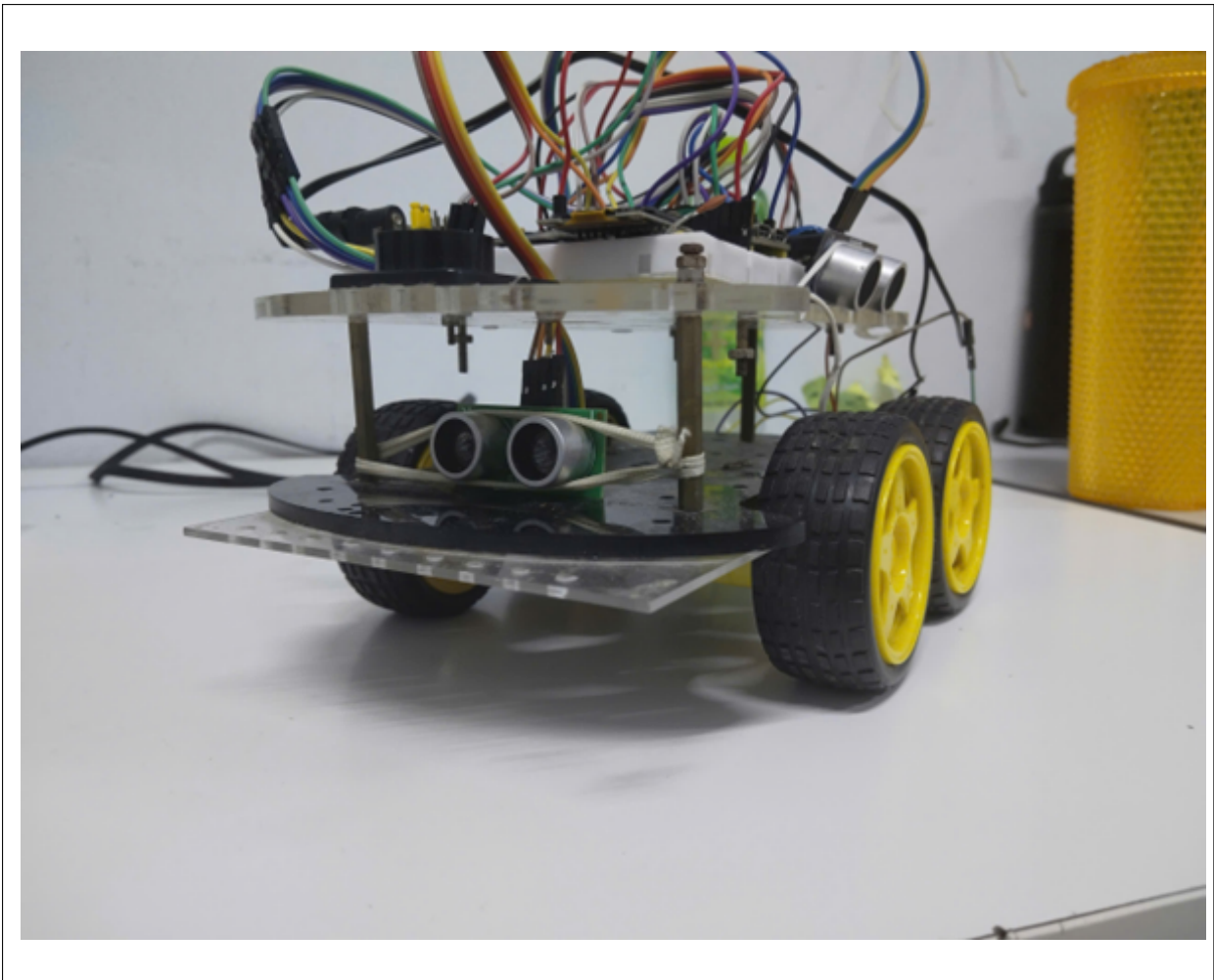
KẾT LUẬN

4.1 Hình ảnh sản phẩm

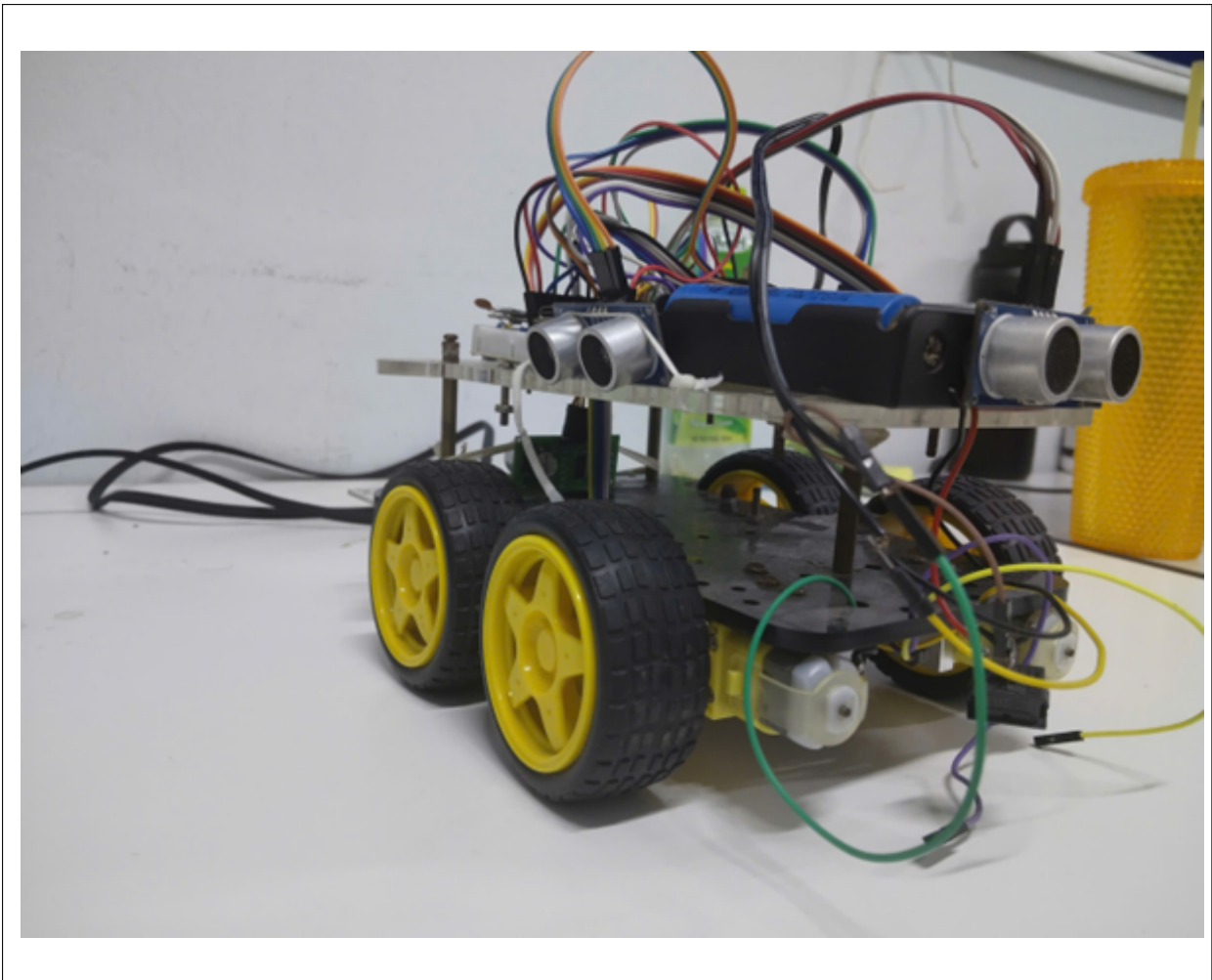




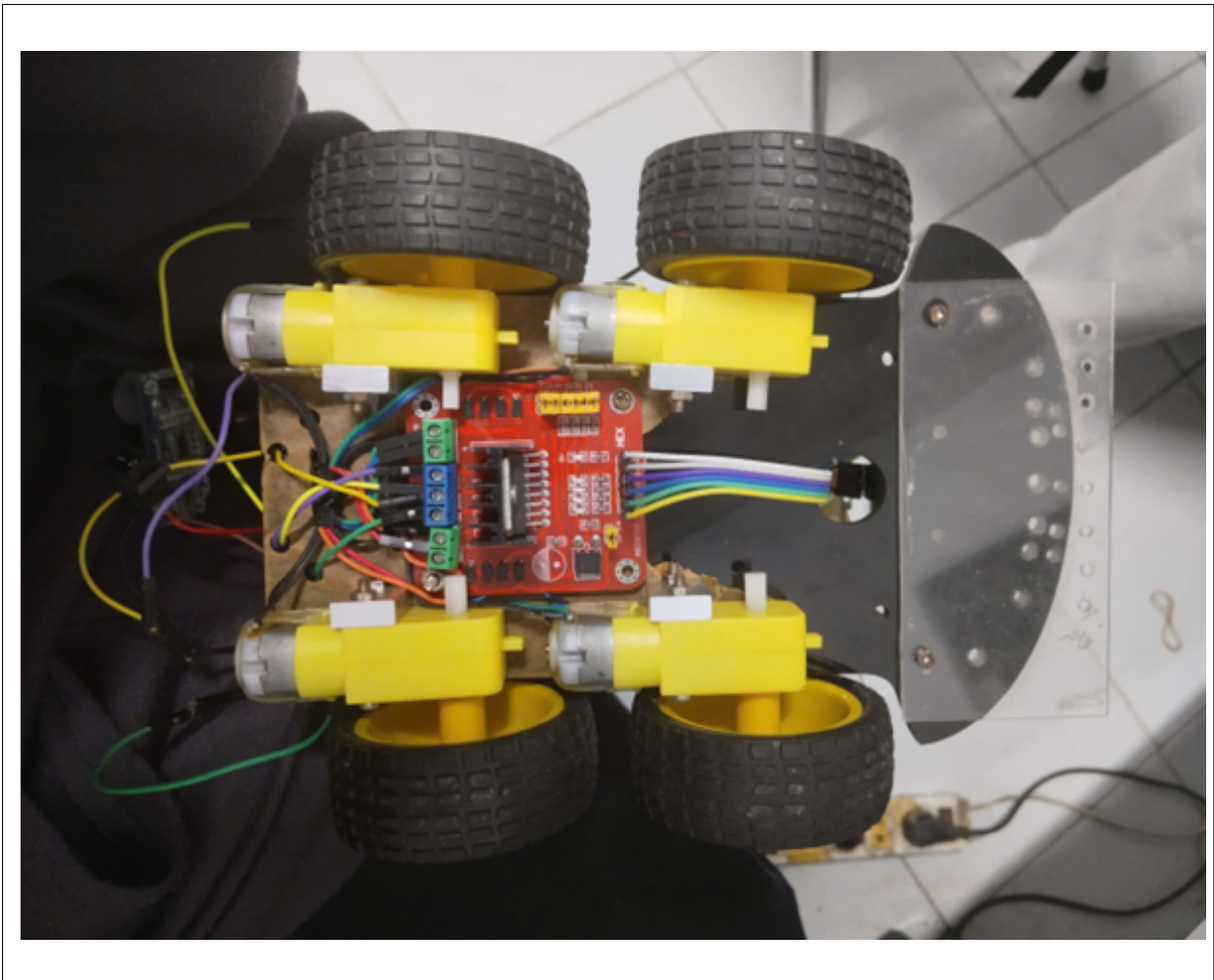
Hình 4.2: Màn OLED hiển thị thông tin



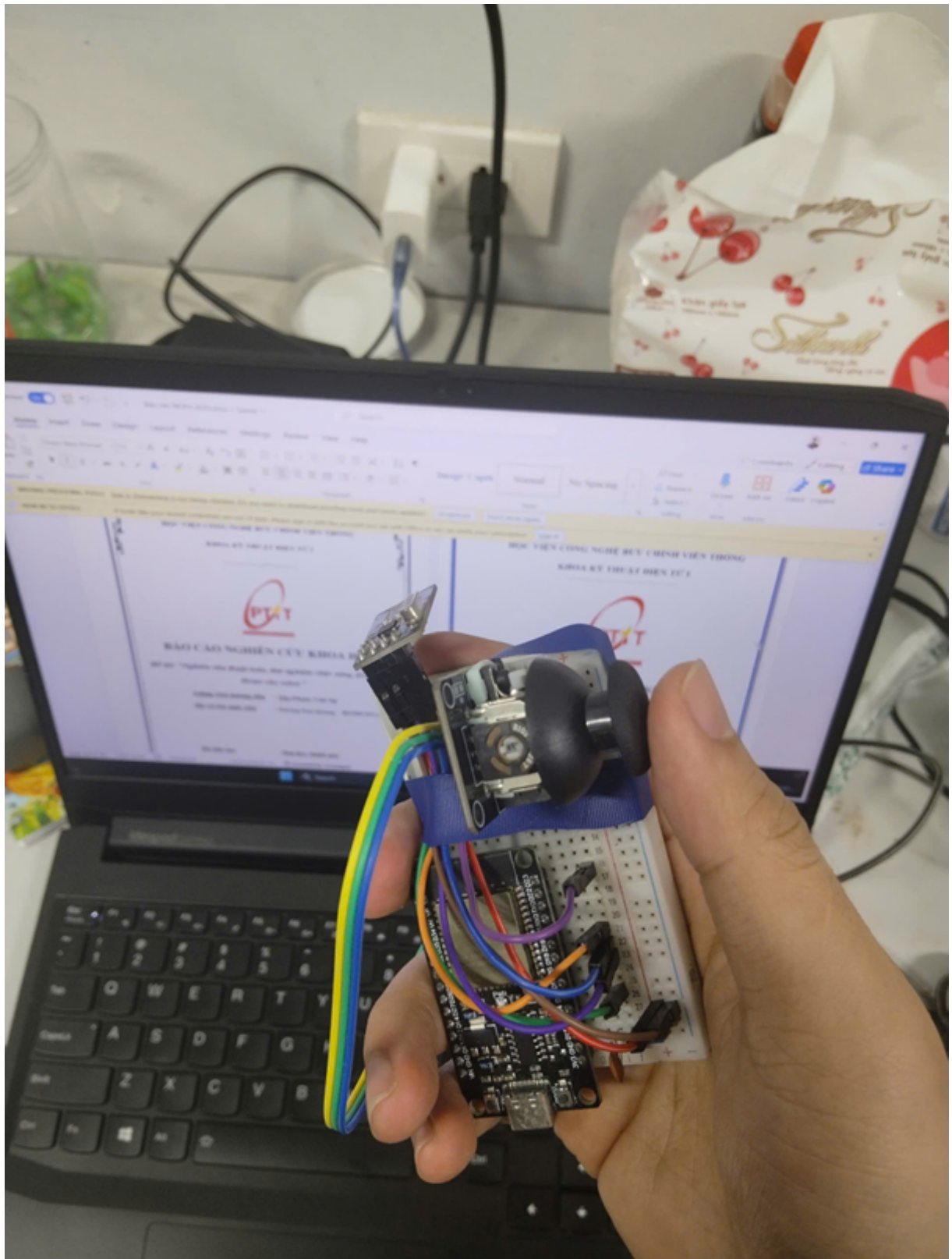
Hình 4.3: Mặt trước Robot



Hình 4.4: Mặt sau Robot



Hình 4.5: Mặt dưới Robot



Hình 4.6: Bộ điều khiển (Controller)

4.2 Kết quả đạt được

Đề tài "Nghiên cứu thuật toán, thử nghiệm chức năng Return To Home cho robot" đã hoàn thành các mục tiêu:

1. **Xây dựng thành công hệ thống RTH** RTH sử dụng GPS và la bàn điện tử, có khả năng tự động quay về điểm xuất phát với độ chính xác 1-5m.
2. **Ứng dụng FreeRTOS** để quản lý đa nhiệm real-time, đảm bảo hệ thống hoạt động mượt mà và ổn định.
3. **Thuật toán PID** cho điều khiển heading, giúp robot giữ hướng chính xác trong quá trình di chuyển.
4. **Tránh vật cản thông minh** bằng 4 cảm biến siêu âm, cho phép robot vượt qua chướng ngại vật trên đường về Home.
5. **Hiệu chuẩn từ kế** tự động, bù nhiễu hard-iron và soft-iron
6. **Bộ lọc nhiễu** Moving Average cho sonar và Circular MA cho heading
7. **Lưu trữ bền vững** Home và calibration vào NVS, robot có thể khởi động lại mà không mất dữ liệu.
8. **Giao tiếp RF** ổn định với controller, hỗ trợ cả lệnh di chuyển và lệnh có xác nhận (ACK).

4.3 Kiến thức thu được

Thông qua quá trình nghiên cứu và thực hiện đề tài, em đã củng cố và mở rộng kiến thức về:

- Lập trình nhúng ESP32-S3 với PlatformIO
- Hệ điều hành thời gian thực FreeRTOS (Task, Queue, Mutex, Watchdog)
- Xử lý cảm biến: GPS, IMU, magnetometer, siêu âm
- Thuật toán điều khiển: PID, State Machine, Haversine, Bearing
- Kỹ thuật lọc nhiễu: Moving Average, Circular Average
- Bù nghiêng (Tilt Compensation) cho la bàn điện tử
- Hiệu chuẩn từ kế (Hard/Soft iron correction)
- Giao tiếp RF với nRF24L01

4.4 Ứng dụng thực tế

Hệ thống có thể áp dụng cho:

- Robot giao hàng tự động
- Drone quay về điểm xuất phát
- Robot tuần tra an ninh
- Xe tự hành trong khuôn viên
- Robot khảo sát địa hình

4.5 Hướng phát triển

4.5.1 Ngắn hạn

- Thêm camera để ghi hình quá trình di chuyển
- Nhận diện vật cản bằng computer vision
- Module SIM để gửi SMS vị trí khi khẩn cấp
- Cảm biến đo dung lượng pin, tự động RTH khi pin yếu
- Encoder bánh xe để odometry, cải thiện dead reckoning

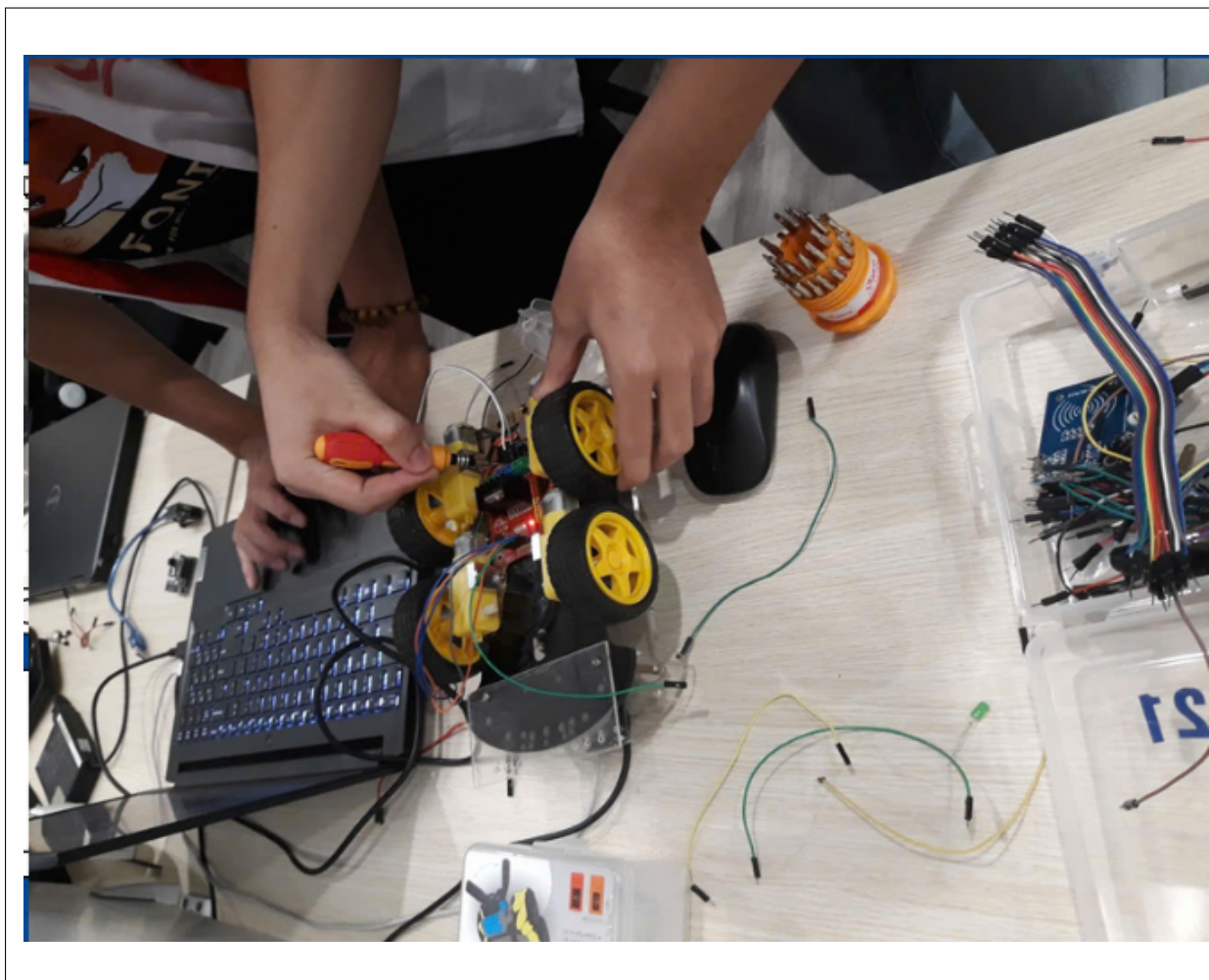
4.5.2 Dài hạn

- Fusion GPS + IMU bằng Kalman Filter/EKF
- Sử dụng GPS RTK cho độ chính xác cm
- SLAM (Simultaneous Localization and Mapping)
- Path planning với A* hoặc RRT
- Multi-robot coordination

4.6 Lời kết

Trong suốt thời gian thực hiện đề tài, nhóm đã được sự giúp đỡ tận tình của thầy Phạm Văn Sự đã góp ý, giúp đỡ nhóm rất nhiều trong việc xây dựng được đề tài này. Là một đề tài khó nhưng nhóm đã thử thách được mình để hoàn thiện đồ án, học được cách làm việc, tổ chức mọi việc khoa học hơn. Mặc dù đã có nhiều cố gắng nhưng khó tránh khỏi những sai sót, nhóm rất mong nhận được sự góp ý từ các thầy cô giáo để có thể hoàn thiện hơn nữa. Xin chân thành cảm ơn! Xin chân thành cảm ơn thầy Phạm Văn Sự đã tận tình hướng dẫn, giúp đỡ em trong suốt quá trình thực hiện đề tài!

4.7 Một số hình ảnh trong quá trình thực hiện đề tài



Hình 4.7: Quá trình thực hiện đề tài



Hình 4.8: Quá trình thực hiện đề tài

TÀI LIỆU THAM KHẢO

1. Espressif Systems, "ESP32-S3 Technical Reference Manual", 2023.
2. TinyGPS++ Library Documentation, <http://arduiniiana.org/libraries/tinygpsplus/>
3. MPU9250 Datasheet, InvenSense/TDK, 2016.
4. QMC5883L Datasheet, QST Corporation, 2016.
5. nRF24L01+ Datasheet, Nordic Semiconductor, 2008.
6. FreeRTOS Documentation, <https://www.freertos.org/>, 2024.
7. PlatformIO Documentation, <https://docs.platformio.org/>, 2024.
8. Haversine Formula, Wikipedia, https://en.wikipedia.org/wiki/Haversine_formula
9. "PID Control Theory", Karl J. Åström and Richard M. Murray, 2008.
10. L298N Dual H-Bridge Motor Driver Datasheet, STMicroelectronics, 2000.
11. SSD1306 OLED Display Controller Datasheet, Solomon Systech, 2008.
12. HC-SR04 Ultrasonic Sensor User Guide, Elecfreaks, 2013.
13. "GPS Navigation using Haversine Formula", Journal of Robotics and Automation, 2019.
14. "Tilt-Compensated Magnetic Compass", Application Note AN4248, Freescale Semiconductor, 2012.
15. Arduino Framework for ESP32, GitHub Repository, <https://github.com/espressif/arduino-esp32>
16. "Digital Compass with Tilt Compensation", AN4248, NXP Semiconductors, 2015.
17. "Magnetometer Calibration", Adafruit Learning System, 2020.